



Desenvolvimento de Sistemas

Testes de *software*

Introdução

Em desenvolvimento de *software* sempre se ouve falar de testes, porém eles nem sempre ganham a importância que deveriam. Mas, afinal, o que são testes?

Pode-se dizer, de maneira informal, que teste é uma forma de fazer uma verificação e/ou validação. E na prática, como se pode definir o teste?

Realizar o processo de testes, ou seja, testar o *software*, é executá-lo a procura de defeitos. O objetivo é encontrar defeitos, todos ou uma boa parte deles.

No conteúdo a seguir, serão explorados conceitos, aspectos e definições sobre testes de *software*, ressaltando sua relevância em um projeto de *software*.



Figura 1 – Representação gráfica dos profissionais de teste de *software*

Fonte: FireWork (2021)

Conceito de teste



Pode-se dizer que teste de *software* é um processo normatizado que usa técnicas de execução de sistema para encontrar defeitos antes que este seja entregue ao cliente/usuário final. Essas técnicas, combinadas com o conhecimento do analista de testes, aumentam a confiabilidade do usuário no *software* que está sendo testado, fazendo com que o usuário tenha a garantia de que o sistema foi validado em relação às suas regras de negócio e de que está atendendo às suas expectativas.



Figura 2 – Ciclo do teste de *software*

Fonte: Senac EAD (2023)

Como é possível ver na imagem, testar é um ciclo. O teste de *software* permite que ele esteja sempre em constante verificação, pois erros são comuns. Porém, essa prática faz com que o *software* possa ser usado constantemente, sem necessidade de parar o trabalho do usuário para que sejam corrigidas as falhas (ou erros) que acabaram passando despercebidos, em razão da falta de teste.

O responsável por esses testes, chamado de analista de testes, tem a função de explorar falhas de um sistema e tentar, por meio de combinações incorretas, expor alguma vulnerabilidade do *software*. Por isso, é possível dizer que as atividades de teste têm característica “destrutiva”, porém esses testes aumentam a segurança e a confiabilidade do sistema, por meio de exposição desses erros e dessas falhas.

Caso de teste



Segundo o glossário da ISTQB (International Software Testing Qualifications Board), um selo de qualidade para testadores de *software*, define-se caso de teste como um conjunto de **valores de entrada, pré-condições de execução, resultados esperados e pós-condições de execução**, desenvolvido para um objetivo particular ou uma condição de teste.

Sabendo disso, como posso usar o caso de teste na prática?

Entenda, agora, um pouco melhor esse conceito.

Valores de entrada são dados que os seres humanos, ou uma máquina, fornecem para o *software*. O *software* recebendo esses dados fará o processamento e dará uma resposta que é comparada com os resultados esperados. Em resumo: você passará um grupo de valores de entrada, o *software* processará e, quando devolver o resultado, você analisará se os resultados esperados são iguais aos resultados processados.

Para que se possa enviar uma informação ao *software* e para que ele possa processar de uma maneira específica, é necessário que o sistema esteja em um estado inicial ou mais avançado, mas que já tenha um escopo desenvolvido, que foi criado com base na regra de negócio. É possível citar algumas **pré-condições** para o teste: que já se tenha um usuário cadastrado, ou que já tenha saldo na conta, ou que já tenha algum requisito específico já configurado no *software*.

Já a **pós-condição** é qualquer tipo de informação que acontecerá no *software* depois que um determinado teste for executado.

Contador:	001
Localização:	Tela de abertura NFE.
Criticidade:	Alta
Objeto de Teste:	Verificar funcionamento da tela de abertura emissão nfe.
Caso de Teste:	Testar o funcionamento do botão "Emissão NF-e e campo fiscal"
Pré - Condição:	1. Operação devidamente cadastrada.
Procedimento:	1. Clicar no campo "Emissão de NF-e" 2. Digite o código da operação ou clique F3 para pesquisar, depois de identificar a operação desejada. Pressione ENTER. 2.1 Digite um carácter, por exemplo, letra "A", no campo operação. 3. Digite o código da empresa ou pressione F3 para pesquisar, depois de selecionada a empresa pressione ENTER. 4. Digite o código do emitente ou pressione F3 para pesquisar, depois de identificado o emitente, pressione ENTER. 5. Clicar no botão "Salvar".
Resultado Esperado:	1. O sistema deverá abrir a tela de abertura para NF-e. 2. O sistema deverá passar para campo EMPRESA carregar dados. 2.1 O Sistema deverá mostrar mensagem de erro. 3. O sistema deverá passar para campo EMITENTE carregar dados. 4. O sistema deverá autenticar o emitente selecionado. 5. O sistema deverá abrir a tela para emissão da nota fiscal eletrônica.

Figura 3 – Exemplo de caso de teste

Fonte: Brasil Escola (2014)

Caso de teste *versus* cenário de teste

Quando se vai realizar testes em alguma aplicação, não se deve confundir as definições de caso de teste e de cenário de teste. Muitas vezes, por não entender muito do assunto, acaba-se achando que os dois são a mesma coisa, porém não são.

Caso de teste é um conjunto de ações executadas para verificar algum recurso ou alguma função da aplicação. **Cenário de teste** é qualquer funcionalidade da aplicação que possa ser testada. O caso de teste foca mais em como testar, já o cenário de teste concentra-se no que testará.

A equipe de controle de qualidade e desenvolvimento da aplicação é a responsável por escrever os casos de teste. Já em um cenário de teste, os responsáveis pela sua criação são os analistas e/ou gerentes de negócios.

Falando em escrita, veja, agora, como escrever um caso de teste.

Escrita do caso de teste

Para realizar a escrita de um caso de teste, deve-se, primeiro, conhecer o que se está testando e, depois disso, criar uma espécie de documento que pode conter todas (ou algumas) as etapas descritas a seguir:

1. ID: é um número único e exclusivo do caso de teste. Isso fará com que cada caso de teste se diferencie do outro.
2. Descrição: é um texto breve, porém adequado ao que se está testando (recurso, unidade, função etc.), com detalhes importantes do caso de teste.
3. Pré-condições: como visto anteriormente, é tudo que deve ser observado antes da execução do teste.
4. Etapas de teste: é muito importante descrever cada passo (cada etapa) usando uma linguagem de fácil compreensão.
5. Resultado esperado/desejado: nesta etapa, são explicados os resultados esperados pelo caso de teste.
6. Pós-condições: aqui, serão descritas as condições que devem ser atendidas após a execução com sucesso do caso de teste.
7. Estado: com base nos resultados obtidos, deve-se mencionar se o *status* do caso de teste é aprovado, caso tenham sido obtidos os resultados desejados, ou reprovado, caso os resultados não tenham sido os esperados.

Exemplificando:

O cliente João realizará uma transferência para Maria (será a favorecida neste exemplo). Não será necessário ir até o banco, pois tudo será feito *on-line*, por meio de um *software* específico. Essa transferência será feita em razão de um processo judicial no qual João deve pagar a quantia de R\$ 800,00 para Maria. No ato da transferência, João tinha essa quantia em sua conta e Maria criou uma conta para esse fim, a qual se encontra sem nenhum dinheiro.

Como essa transferência se deve a um processo judicial, João terá de realizá-la usando um *software* específico. Nesse sistema, ele informará os próprios dados de *login*. Após isso, irá na aba de **Transferência Judicial**, informará os dados da conta de Maria e confirmará a transferência. Feito isso, João sairá do sistema, para que o seu usuário não fique “logado”. Se tudo der certo, Maria ficará com o saldo de R\$ 800,00 e o processo judicial terá sido executado com sucesso.

Com base no cenário anterior, é possível montar um caso de teste, conforme o exemplo a seguir.

Exemplo de caso de teste**ID:** 001**Descrição:** Transferência de valor com saldo positivo**Pré-condições de execução:**

- O cliente João precisa ter saldo de R\$ 800,00.
- A favorecida (pessoa que receberá o dinheiro) precisa ter saldo de R\$ 0,00.

Valores de entradas:

- Maria como favorecida
- R\$ 800,00 como valor de transferência

Etapas de teste:

Etapa	Descrição	Resultado esperado
1	Faça <i>login</i> com os dados do cliente	Tela secreta apresentada
2	Inicie uma transferência de valores	Foco no campo Favorecido
3	Informe R\$ 800,00 para Maria	Mensagem de confirmação é apresentada
4	Confirme a transferência	Mensagem de sucesso na transferência é apresentada
5	Faça <i>logout</i> do sistema	Tela de <i>login</i> é apresentada

Resultados esperados:

- Sucesso ao transferir a quantia

Pós-condições de execução:

- Novo saldo do cliente que realizou a transferência será de R\$ 0,00.
- Novo saldo da favorecida (Maria) será de R\$ 800,00.

Estado: Aprovado

Tabela 1 – Exemplo de caso de teste

Fonte: Senac EAD (2023)

Veja, a seguir, outro exemplo.

Gustavo finalizou a programação de uma tela de cadastro que será implementada em sistema *desktop* já existente. Na empresa onde esse sistema *desktop* é usado, somente alguns usuários poderão preencher essa tela de cadastro e salvar os dados. Foi solicitado que um usuário com a devida permissão faça o cadastro e tente salvar. O usuário em questão acessou o sistema, colocou os próprios dados de *login*, acessou a tela de cadastro e conseguiu realizar um cadastro com sucesso.

Baseando-se no cenário anterior, é possível montar um caso de teste, conforme o exemplo a seguir.

Exemplo de caso de teste**ID:** 002**Descrição:** Preenchimento e salvamento de dados em uma tela de cadastro**Pré-condições de execução:**

- O usuário precisa ter permissão para acessar a tela de cadastro.
- O *login* e a senha do usuário precisam ser válidos.

Valores de entrada:

- Usuário com acesso à tela de cadastro.
- Usuário fornece os dados de *login* dele.

Etapas de teste:

Etapa	Descrição	Resultado esperado
1	Faça <i>login</i> com os dados do cliente	Direcionamento para o menu de opções
2	Acesse a tela de cadastro	Tela de cadastro é apresentada
3	Preencha todos os campos do cadastro	Os dados estão corretos
4	Salve o cadastro	Mensagem de cadastro realizado com sucesso é apresentada
5	Faça <i>logout</i> do sistema	Tela de <i>login</i> é apresentada

Resultados esperados:

- Cadastro realizado com sucesso.

Pós-condições de execução:

- Usuário autorizado consegue preencher a tela de cadastro e salvar.

Estado: Aprovado

Tabela 2 – Exemplo de caso de teste

Fonte: Senac EAD (2023)

Pratique por meio do desafio a seguir.

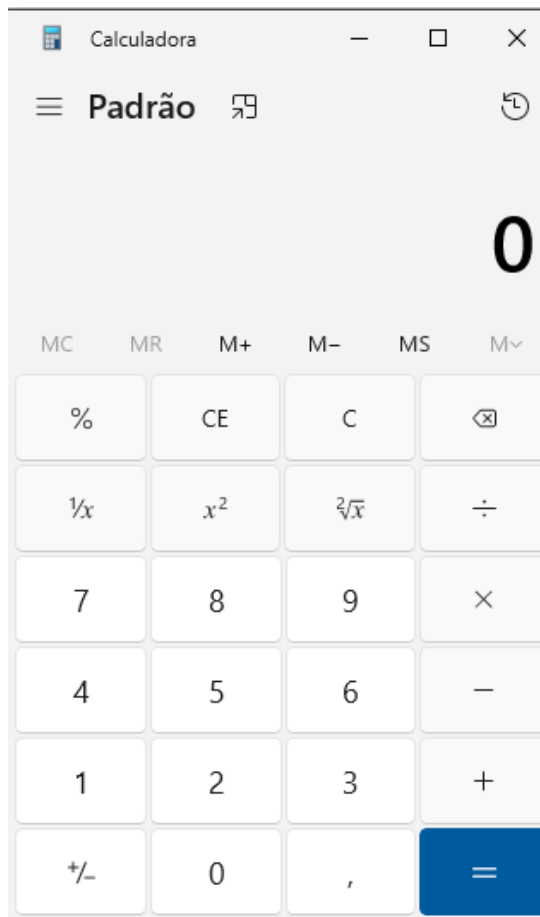


Figura 4 – Calculadora do Windows

Fonte: Senac EAD (2023)

A equipe de teste da empresa Senac Tecnologia foi contratada para testar um *software* que simula uma calculadora virtual. Devem ser testadas todas as funcionalidades apresentadas pela calculadora para efetuar os cálculos (não precisa testar teclas nem abas de função). A equipe deverá preparar um documento de teste contendo pelo menos três casos de teste.

Erro, defeito e falha: entendendo a diferença

Antes de conceituar essas três palavras, veja um pequeno cenário criado em que, na prática, erro, defeito e falha acontecerão.

Jorge foi contratado por uma empresa para criar um cadastro de produtos dentro de um sistema que está em andamento. Ao codificar, Jorge se esqueceu de alguns detalhes muito importantes em seu código, mas, mesmo assim, enviou suas linhas de código para os outros desenvolvedores, que implementaram no sistema. Logo após isso, o sistema apresentou **erro** ao cadastrar, em razão do código incompleto. Os desenvolvedores não sabiam o que fazer e executaram o sistema, que apresentou um grande **defeito**, em razão do erro de Jorge. Após essa execução não ter sido bem sucedida, o sistema **falha**.

Resumindo:



- ◆ **Erro:** foi cometido por Jorge, ao raciocinar o que precisava fazer.
- ◆ **Defeito:** foi inserido pelo código errado de Jorge.
- ◆ **Falha:** foi causada porque o sistema passou pelo fluxo defeituoso.

A seguir, veja as definições formais dessas palavras.

Erro: estado inconsistente, observado em momento de execução, de um produto ou sistema de *software*. Esse erro pode ou não causar uma falha.

Defeito/bug: passo, processo ou definição de dados incorretos; ou, ainda, qualquer problema existente no *software* ou no sistema sob teste que possa causar sua falha em atender às expectativas do usuário. Em outras palavras, um defeito é uma fonte potencial de insatisfação com o produto.

Falha: estado inesperado do produto ou do sistema de *software*, ocasionado por um erro.



Figura 5 – Explicação de erro, defeito e falha

Fonte: adaptado de Jordan (2018)

Plano de teste

O plano de teste contém os casos de uso, as telas e o projeto que serão testados, os responsáveis pelo teste, por elaborar roteiros e criar ambiente, os erros que podem ser encontrados, os riscos do processo etc. É o planejamento inicial do teste sobre a demanda enviada para a equipe.

Dependendo do tamanho do projeto de desenvolvimento, é possível ter diversos planos de teste para um mesmo projeto de teste de *software*. Esses planos podem ser separados por módulos, funcionalidades ou requisitos, ou, ainda, por um grupo de cada um desses elementos.

Além disso, os planos podem também ser classificados por nível de teste: unitário, integração, sistema e aceitação. Quando o sistema for pequeno, não há necessidade da criação de vários planos de teste, embora, mesmo nesses casos, possa ser razoável ter os planos de teste separados por nível de teste.

Além de definir a agenda de teste e os procedimentos necessários para testar o produto, o plano de teste define a infraestrutura necessária. Nesse sentido, no plano de teste, encontram-se também as anotações sobre os recursos humanos, de *hardware* e de *software* necessários. Essas estimativas são úteis principalmente para os gerentes de projetos e configuração que são responsáveis por garantir que esses recursos estarão disponíveis para a execução dos testes.

Um plano de testes menos formal pode ser utilizado em pequenos sistemas, mas deve ser utilizado um documento formal para auxiliar o planejamento do processo de teste. Os planos de teste não são documentos estáticos, pois evoluem durante o processo de desenvolvimento, mas existe uma padronização dos planos de teste que está descrita na norma IEEE 829 4.1.4. Essa sigla refere-se ao Instituto de Engenheiros Eletricistas e Eletrônicos.

Você pode conferir mais detalhes sobre esses documentos no conteúdo **Plano de teste** desta unidade curricular.

Teste estático e teste dinâmico

As atividades para avaliação da qualidade de *software* podem ser divididas em duas categorias amplas: teste estático e teste dinâmico.

Teste estático

Como o termo “estático” sugere, baseia-se no exame de uma série de documentos, nomeadamente documentos de requisitos, *software*, modelos, documentos de *design* e código-fonte. O teste estático tradicional inclui revisão de código, inspeção, passo a passo, análise de algoritmo e prova de veracidade. Não envolve a execução real do código em desenvolvimento. Em vez disso, ele examina o código e os motivos de todos os comportamentos possíveis que podem surgir durante o tempo de execução.

Pode-se dizer que, quando se está olhando um código e analisando-o, um teste e/ou uma análise estática estão sendo feitos. Nessa análise, é possível realizar algumas verificações, como:

- ◆ O código foi documentado?
- ◆ As variáveis, as constantes e as classes estão com uma nomenclatura aceitável?
- ◆ O código está organizado e é de fácil compreensão?
- ◆ O programador, ao codificar, obedece à arquitetura do sistema?

Teste dinâmico

O teste dinâmico de um sistema envolve a execução do programa para expor suas possíveis falhas. As propriedades comportamentais e de desempenho do programa também são observadas. Os programas são executados com valores de entrada típicos e cuidadosamente escolhidos.

Esse teste acontece dentro do ambiente de desenvolvimento e envolve verificações como:

- ◆ Entrar e sair do sistema.
- ◆ Inserir *login* e senha e verificar se autorizará.
- ◆ Abrir as opções desejadas no menu.
- ◆ Clicar em determinada opção e o *software* direcionar a ela.

Portanto, nos testes, alguns comportamentos representativos do programa são observados e é possível chegar a uma conclusão sobre a qualidade do sistema.

Ao realizar testes estáticos e dinâmicos, os profissionais desejam identificar o máximo de falhas possível, para que estas sejam corrigidas em um estágio inicial de desenvolvimento do *software*. A análise estática e a análise dinâmica são complementares por natureza e, para ter melhor eficácia, ambas devem ser realizadas de forma repetida e alternada.

Critério de aceitação

Antes de realizar um teste de aceitação (tipo de teste que acontece geralmente antes da implantação do *software* e tem como objetivo verificar se ele está pronto, aceito pelo cliente, e já pode ser utilizado para a sua finalidade), é preciso definir, de forma clara, quais critérios serão utilizados. Tais critérios são divididos em: critério de aceitação funcional e critério de aceitação não funcional.

O **critério de aceitação funcional** diz respeito ao comportamento do sistema, em como ele ajudará os seus usuários a realizar determinado trabalho. Refere-se às funções ou aos recursos que o sistema oferece.

Já o **critério de aceitação não funcional** especifica todos os requisitos. Ele avalia sobre como o sistema funciona sem levar muito em consideração os códigos, e sim os aspectos de acessibilidade, a facilidade de uso, a sua segurança e privacidade, a velocidade entre as telas, a confiabilidade, entre outros aspectos.

Como definir os critérios de aceitação

Após entender os conceitos dos critérios de aceitação, você aprenderá como definir esses critérios, que incluem os requisitos de desempenho e as condições essenciais que serão atendidas antes de se realizar a entrega do projeto. Os critérios determinam sob quais situações o cliente aceitará o resultado final do projeto. Assim, é possível medir e provar ao cliente que o trabalho está completo.

Algumas técnicas podem ser usadas para definir critérios de aceitação. Para funcionalidades em que é possível definir um cenário de uso, o esquema “dado que/quando/então” (*given/when/then*) pode ser útil.

- ◆ “Dado que” alguma pré-condição
- ◆ “Quando” uma ação é executada
- ◆ “Então” espera-se algum resultado

Veja, a seguir, um exemplo dessa técnica aplicada a um caso de uso para um caixa eletrônico.

História de usuário:

Como usuário, quero ser capaz de solicitar o saque de um terminal de caixa eletrônico de maneira que eu possa receber o dinheiro da minha conta bancária rapidamente em diferentes locais.

Cenário 1: Solicitar o saque de uma conta com crédito

Dado que	A conta possui saldo
E	O cartão é válido
E	O caixa eletrônico tem notas
Quando	O usuário solicita o saque
Então	Assegure-se que a conta seja debitada no valor solicitado
E	Assegure-se que as notas sejam disponibilizadas pela máquina
E	Assegure-se que o cartão é liberado ao usuário

Tabela 3 – Exemplo do esquema “dado que/quando/então”

Fonte: Senac EAD (2023)

Modernize a história de usuário exemplificada anteriormente trocando caixa eletrônico por Pix. Considere o cenário em que o usuário vai transferir dinheiro a outra pessoa usando uma chave Pix.

Em outros casos mais técnicos, em que seja difícil aplicar o esquema “dado que/quando/então”, como a interação com uma interface gráfica, pode ser necessário usar um esquema orientado a regras. Nesse formato, usa-se geralmente uma *checklist* de regras que descrevem o comportamento de um sistema ou de uma funcionalidade específica. Com base nessas regras, pode-se desenhar cenários mais específicos.

Veja, no exemplo a seguir, uma história de usuário para um sistema *web* que faz busca por hotéis.

História de usuário:

Como usuário, quero ser capaz de usar um campo de pesquisa no sistema para encontrar opções de hotel, informando cidade, nome ou rua.

Critérios básicos de aceitação para busca:

- ◆ O campo de busca está posicionado no topo da tela.
- ◆ A busca inicia ao clicar no ícone **Lupa**.
- ◆ O campo está vazio.
- ◆ A busca é realizada se um usuário informar uma cidade, um nome de hotel, uma rua ou todas essas informações combinadas.
- ◆ A busca ocorre em português.
- ◆ O usuário não pode digitar mais que 200 caracteres.

Tabela 4 – Exemplo de busca

Fonte: Senac EAD (2023)

Outros formatos próprios para escrita de critério de aceitação podem ser aplicados, como tabelas com valores possíveis para uma funcionalidade ou texto explicativo do que deve ou não ocorrer em um cenário de uso específico.

Veja, a seguir, alguns exemplos de critérios de aceitação não funcionais.

1. **Requisito:** O sistema deve ser capaz de processar a listagem de dados automaticamente.
Critério de aceitação: A falha no processamento de dados não pode ultrapassar 5%;
2. **Requisito:** O processo de *backup* e restauração devem estar em perfeito funcionamento.
Critério de aceitação: O processo de *backup* e restauração foi testado com sucesso.
3. **Requisito:** A funcionalidade de *login* não deve permitir que um usuário não cadastrado no banco de dados faça *login* no sistema.
Critério de aceitação: Somente usuários autenticados (cadastrados no banco de dados) acessam o sistema.

Características dos critérios de aceitação

- Os critérios de aceitação devem ser fáceis de serem testados pelo usuário. Seus resultados devem ser de simples interpretação. Dentro do teste de aceitação, as respostas esperadas seriam: sim ou não, aprovado ou reprovado, aprovado ou resultados inconclusivos etc.
- Os critérios de aceitação devem ser compreensíveis e resumidos. Não há necessidade de uma extensão documentação; a finalidade é tentar ser o mais claro e breve possível.
- Os critérios de aceitação devem ser compreendidos por todos os usuários.
- Os critérios de aceitação devem fornecer uma perspectiva do usuário. Eles farão com que o problema seja percebido pelo ponto de vista do usuário, e isso com certeza trará resultados positivos para o sistema.

Artefatos de testes

Cada empresa e cada metodologia definidas no começo do processo de desenvolvimento de *software* pedem um tipo de artefato para resolver as necessidades do projeto/sistema. Não há um documento padrão seguido por todas as equipes e empresas do mundo. É preciso saber como o processo de teste na empresa foi montado para, então, ver quais documentos devem ser produzidos.

O artefato de teste é uma parte integrante do teste do *software*, e o conjunto de documentos que o compõe fazem parte do ciclo de vida do teste do *software*.

Sabendo disso, há alguns tipos de documentos que podem/devem ser usados para entender se a necessidade do projeto/sistema foi, de fato, resolvida. Veja, a seguir, informações sobre alguns desses documentos.

Estratégia de teste

Geralmente é desenvolvida pelo gerente de projeto. Uma estratégia de teste é um documento que lista os detalhes sobre como todo o projeto prosseguirá. Isso inclui requisitos e recursos para o projeto, estratégias de teste envolvidas, *design* de teste, fases incrementais envolvidas, processos de comunicação com o cliente etc. É o resumo do documento que descreve a abordagem de teste do ciclo de desenvolvimento de *software* e lista como alcançar o resultado esperado usando os recursos disponíveis.

Para desenvolver essa estratégia, há vários pontos que devem ser considerados. A seguir, veja alguns deles.

- Qual é o principal objetivo do teste e o motivo pelo qual você deseja realizá-lo?
- Quais são as diretrizes que devem ser seguidas para a realização de testes?
- Quais são todos os requisitos necessários para o teste, como requisitos funcionais, cenários de teste, recursos etc.?
- Quais são as funções e as responsabilidades de cada função e cada gerente de projeto para concluir o teste?
- Quais são os diferentes níveis de teste?
- Qual será a principal entrega desse teste?
- Quais riscos existem em relação aos testes com os riscos do projeto?
- Existe algum método para resolver problemas que possam surgir?

Plano de teste

Como citado anteriormente, é um documento de planejamento do processo de teste do *software*.

Cenário de teste

É uma condição criada para realizar testes de ponta a ponta bem-sucedidos. Vários casos de teste vêm em um cenário de teste.

Às vezes, também se refere a uma condição ou uma possibilidade de teste. Um cenário de teste resolve o dilema da aplicação do *software* na vida real.

Casos de teste

Este artefato de teste foi citado dentro deste conteúdo.

Dados de teste

Para executar os casos de teste, o engenheiro de controle de qualidade precisa de alguns dados sobre os quais o caso de teste foi aprovado ou reprovado para decidir. Para isso, é criado um documento que contém todos os dados de teste necessários para a execução dos casos de teste criados.

Relatório de defeito/*bug*

Um relatório de defeitos é um documento criado após a execução de todos os casos de teste e o registro dos resultados. Ele lista todos os *bugs* encontrados durante o teste. Isso torna mais fácil, para a equipe de desenvolvimento, a correção de *bugs*.

Relatório de resumo de teste

Ao final de cada um dos ciclos de teste completos, um relatório final é criado. Esse relatório inclui os detalhes de todo o processo de teste. Por exemplo: detalhes sobre casos de teste, resultados obtidos, *bugs* corrigidos, *status* atual do projeto etc.

Esse documento é importante e deve ser apresentado às partes interessadas para mantê-las atualizadas sobre o andamento do projeto. É um documento formal e tudo está claramente documentado para que qualquer pessoa possa entender o relatório.

Guia do usuário

Com todo *software* desenvolvido e pronto para ser implantado no mercado, o guia do usuário é criado ao final. Esse guia é útil para usuários finais e fornece informações detalhadas sobre o *software* e sobre o uso deste.

Encerramento

Testar um *software* é muito importante, porém ainda mais importante é saber o que testar. Um *software* sem defeito não significa que está perfeito, pode significar também que não foi totalmente testado ou foi testado de maneira errada.

Dentro de qualquer projeto, é muito importante sempre verificar a documentação, para compreender melhor o *software* e também o usuário final. Dentro dos testes de *software*, isso é essencial, pois é o usuário final quem utilizará o sistema no dia a dia, então é quem pode melhor dizer se há erros ou não.



Desenvolvimento de Sistemas

Estratégias e níveis de teste

Para o bom andamento de um projeto, é essencial que exista planejamento na realização de algumas funções, e com a etapa de teste isso não é diferente. Para o sucesso do projeto, é necessária a existência de um roteiro para a realização das tarefas, definindo questões como tempo, esforço a ser realizado, recursos necessários e passos a serem executados.

Estratégias de teste: preventiva e reativa

A estratégia deve nortear alguns requisitos que são necessários para o acompanhamento, portanto deve ter os seguintes itens: indicar os objetivos do teste que está sendo realizado; indicar quais serão os papéis e as pessoas envolvidas no plano de teste; e determinar qual é o padrão que as documentações seguirão, por exemplo, como será desenvolvido o documento padrão de relatório de defeitos. Além de determinar quais serão os tipos de testes que cada nível terá, é preciso especificar quais serão as técnicas de testes aplicadas, assim como determinar a utilização de testes automáticos/manuais.

Requisitos de cobertura e métrica também devem estar especificados no documento de estratégias, portanto deve conter a quantidade de *bugs* esperados e definir a quantidade de *bugs* por linha e a quantidade de vezes que uma tarefa retorna para o desenvolvimento. Também deve especificar quais serão as ferramentas utilizadas para o acompanhamento dos defeitos, ou seja, o uso de ferramentas como Jira e Mantis, e definir como deve ser realizada a descrição dos problemas.

Ao criar a estratégia, devem-se considerar também os ambientes de testes que serão utilizados (desenvolvimento, homologação e produção) e a definição de quais relatórios serão realizados e quais devem ser entregues.

Esses itens são os sugeridos pela norma do IEEE (Instituto de Engenheiros Eletricistas e Eletrônicos) que especifica o padrão a ser seguido, porém algumas empresas realizam adaptações conforme a sua realidade, considerando técnicas e metodologias de desenvolvimento aplicadas.

Geralmente a estratégia de teste fica em um *wiki*, em uma página *web* interna, em razão da fácil manutenção e do fácil acesso a todos da instituição, pois manter outras formas de documento gerará entraves para acesso, como a necessidade de licenças para realizar edições ou até mesmo a necessidade de *download* de arquivo toda vez que a estratégia passar por alterações.

Alguns fatores são essenciais para a criação da estratégia. O primeiro passo é compreender a realidade de desenvolvimento da instituição, ou seja, ambientar-se às ferramentas que a empresa usa, compreendendo como acontecem os processos e seus fluxos e inteirando-se das tecnologias que são utilizadas (linguagem de programação e teste com suas métricas de cobertura). O segundo fator de grande importância é conhecer a quantidade de ambientes de testes que estarão disponíveis e se serão utilizados ambientes gerais ou ambientes específicos para cada tipo de teste, por exemplo, o teste de carga.

Outro fator preponderante é conhecer quais são as áreas mais importantes do sistema que está sendo desenvolvido. Desse modo, deve-se compreender quais pontos, caso sejam liberados com falhas, podem ser fatais ao projeto, significando uma mancha irreversível para a imagem do cliente.

Um exemplo para uma situação dessas seria projetar um *software* para um sistema hospitalar que auxiliará médicos em diagnósticos de exames de alto grau de risco. Nesse caso, uma falha poderia trazer sérias consequências, pois geraria um diagnóstico que poderia colocar um paciente em grave risco de vida. No caso de ocorrência de uma fatalidade, isso afetaria a integridade dessa instituição junto à opinião pública, acarretando uma marca negativa tanto para o cliente quanto para a empresa de desenvolvimento. Além disso, possíveis problemas jurídicos poderiam ser desencadeados.

Outro fator essencial é conhecer a pirâmide de teste automatizado, para iniciar pelas bases da pirâmide, verificando se já existe alguma métrica de cobertura ou, no caso de estar sendo realizada a primeira bateria de testes em um *software* novo, criando esses parâmetros. Após essas definições serem realizadas, o próximo passo é subir para os degraus mais altos da estrutura e, por fim, verificar quais são os testes não funcionais que devem ser realizados.

Esses são os fatores que devem ser considerados na hora da criação de estratégias, aplicando-os também para atualizações futuras.

Existem algumas abordagens que podem ser aplicadas, como no caso da abordagem preventiva ou reativa, que considera o período em que os testes devem iniciar, e da abordagem analítica ou heurística, que considera a base de dados conhecida.

Estratégia preventiva

A abordagem preventiva considera a participação da equipe de teste desde o momento em que o projeto é iniciado, minimizando, dessa forma, os custos, pois, ao especificar casos de teste e aplicá-los, possíveis falhas são encontradas com antecedência no processo. Assim, evita-se que ocorra retrabalho pela equipe de desenvolvimento, otimizando o tempo para o desenvolvimento de tarefas essenciais.

Um exemplo de projeto em que podem ser utilizadas estratégias preventivas é o desenvolvimento de uma aplicação Java para a organização de tarefas pessoais, em que, durante o processo de desenvolvimento, testes unitários foram produzidos para serem executados com periodicidade, garantindo a funcionalidade das classes e dos métodos da aplicação. Também foram produzidos testes de integração para cobrir a interação correta entre as classes e os métodos, validando entradas de dados e inclusão de operações de adição, edição e remoção de tarefas.

Estratégia reativa

Em uma abordagem de teste reativa, a equipe de teste participará do projeto já durante o desenvolvimento, ou seja, quando funcionalidades já foram desenvolvidas e muitas delas integradas, o que, após a realização dos testes e com o encontro de falhas, pode significar em retrabalho e onerar custos.

Um exemplo de projeto em que uma estratégia de teste reativa foi aplicada é o desenvolvimento de um aplicativo de gerenciamento de finanças pessoais. Durante o processo de desenvolvimento, foram realizados testes reativos para garantir a funcionalidade e a correção de erros no aplicativo. Isso incluiu testes de unidade para verificar o comportamento de cada componente individualmente, testes de integração para verificar a interação entre componentes e testes de sistema para verificar a funcionalidade geral do aplicativo.

Abordagem analítica e heurística



Na abordagem analítica, são utilizados dados para realizar o planejamento dos testes. É adicionada uma métrica para finalizar os testes, por exemplo: os testes são interrompidos quando chegarem a uma cobertura de 95%. Dessa forma, os testes são realizados considerando parâmetros como custo e tempo para definir a quantidade e a intensidade dos testes.

A técnica de análise heurística consiste na avaliação da usabilidade, para identificar problemas de usabilidade em sistemas interativos, como *softwares* e aplicações. Nessa abordagem heurística, os testes são realizados considerando o conhecimento e o domínio da equipe de teste, conforme os requisitos especificados no projeto. Após a realização dos testes, é fornecido um *feedback* sobre os problemas com a interface e a experiência do usuário.

Níveis de teste: unitário, sistema, integração e aceitação

A realização de testes é uma etapa importante para assegurar a qualidade de um *software*, portanto é necessária a realização de gerenciamento e organização dos testes, o que abrange a definição de níveis de testes a serem executados.

Sendo o conjunto de tarefas relacionadas ao gerenciamento e à organização dos testes, os níveis de testes consistem em: componentes, integração, sistema e aceite. Cada tarefa é realizada em um determinado momento do desenvolvimento do projeto, gerando uma esfera de teste, podendo ser realizado em unidades individuais ou componentes até englobar sistemas totalmente concluídos, tendo relação com diferentes tarefas nas etapas do desenvolvimento de *software*.

Cada nível de teste deve ter um ambiente condizente com a sua realidade. É possível citar como exemplo o teste de aceitação, em que o ideal é a realização com aspectos semelhantes ao ambiente de produção em que o sistema será executado. Já um teste de componente geralmente é realizado no ambiente de desenvolvimento.

Um nível de teste deve ter as seguintes propriedades: objetivos específicos; bases de teste diferentes para serem utilizadas para criar casos de testes; objeto de teste, que é aquilo que será testado; defeitos e falhas típicas e abordagens; e responsabilidades específicas.

O teste de componentes também é conhecido como teste de unidade. O teste de integração tem duas partes, sendo dividido em: integração entre componentes e integração entre sistemas. Já o teste de aceite ou aceitação pode ser: aceite de usuário, aceite operacional, aceite contratual e alfa/beta teste.

Teste de componente (unidade)

Será um teste realizado no menor fragmento do sistema, podendo ser realizado em uma função, uma classe ou um componente do *software*. Portanto, é necessário ter acesso ao código-fonte, sendo realizado pelo próprio desenvolvedor, de forma isolada de outras partes do sistema, como ao testar uma função que calculará o valor a ser pago em um período dentro de um estacionamento. A função deve executar outra função ao final do pagamento, a qual emite a nota fiscal. O teste deve ser executado apenas na função de cálculo do pagamento, simulando a sequência da execução da outra função. Isso é uma característica essencial do teste de unidade. Outra característica marcante é a rapidez com a qual o teste pode ser executado. Em metodologias ágeis, ele é realizado antes do desenvolvimento do código, conhecido como TDD, que significa *test driven development* (desenvolvimento guiado por teste, em português). Outra propriedade desse tipo de teste é a automatização dele.

Esse teste tem como objetivos reduzir risco, considerar aspectos de requisitos funcionais e não funcionais dos componentes esperados e especificados, gerar uma relação de confiança na qualidade do componente, buscar defeitos no componente e evitar que os problemas se expandam para níveis mais altos de teste.

Para a base de teste é utilizado o projeto detalhado, o código e as especificações dos componentes. O objeto de teste é a própria estrutura de código, considerando classes e módulos de bancos de dados.

São consideradas falhas típicas funções que não têm lógica e/ou estrutura de desenvolvimento de códigos corretos e que destoam do que foi informado na especificação do projeto. A velocidade de correção do teste se dá em razão de a realização do teste ser feita pelo próprio desenvolvedor, que, ao perceber a falha, já realiza a correção, sem a necessidade de abrir um chamado que parará em uma lista de eventuais ajustes.

Considere o seguinte código:

```
public class Calculadora {  
    public int soma(int a, int b) {  
        return a + b;  
    }  
  
    public int subtrai(int a, int b) {  
        return a - b;  
    }  
  
    public int multiplica(int a, int b) {  
        return a * b;  
    }  
  
    public int divide(int a, int b) {  
        if (b == 0) {  
            throw new IllegalArgumentException("Não é possível dividir por zero.");  
        }  
        return a / b;  
    }  
}
```

Agora, pense no seguinte exemplo. Ao criar uma classe para realizar as quatro operações básicas da matemática, serão realizados testes para verificar se a soma, a subtração, a multiplicação e a divisão estão retornando o valor correto do cálculo e se na divisão, ao informar um valor 0, uma exceção é lançada.

Teste de integração

No teste de integração, assim como no teste de unidade, é necessário acesso ao código-fonte, de modo a realizar testes baseados na integração entre componentes ou em sistemas, se for o caso. Desenvolvedores geralmente realizam testes nos componentes desenvolvidos, enquanto os testadores realizarão os testes caso exista a integração entre um ou mais sistemas.

O teste de integração de *software* é um tipo de teste que verifica se as diferentes partes de um sistema de *software* estão funcionando corretamente juntas, isto é, se há uma integração adequada entre as partes. Ele é importante porque permite identificar problemas que não podem ser detectados por meio de testes unitários isolados, por exemplo, problemas de comunicação entre componentes, problemas de consistência de dados entre módulos, entre outros.

O objetivo do teste de integração é assegurar que o sistema como um todo funciona corretamente, antes do lançamento em produção. Ele é realizado após o término dos testes unitários e geralmente envolve a combinação de vários módulos ou componentes para formar o sistema completo.

É importante ressaltar que o teste de integração é subdividido em duas partes. A primeira é o teste de integração de componentes, que é realizado após o teste de componentes, tendo o objetivo de testar as interações e interfaces nos componentes integrados, de forma automatizada na maioria das situações. Em um desenvolvimento iterativo e incremental, esses testes ocorrem na execução de integração contínua.

A segunda parte é a integração de sistema, voltada para a interação entre interfaces do sistema, considerando inclusive interfaces que possam ser fornecidas por sistemas de terceiros, por exemplo, um sistema que realizará vendas e consumirá uma API (*application programming interface*, ou, em português, interface de programação de aplicação) de transações bancárias para realizar a funcionalidade de pagamento.

Como base de teste pode ser utilizado o próprio *software*, o diagrama de sequência, os protocolos de comunicação e as especificações de interfaces, os casos de uso e a arquitetura em nível de componente ou sistema. Como objetos de testes, tem-se: subsistemas, bancos de dados, infraestrutura, interfaces e APIs.

Os defeitos típicos que podem ser encontrados no teste de integração entre componentes são: ausência de dados, dados errôneos e desenvolvimento de código incorreto. O uso de bibliotecas incompatíveis pode gerar falhas por falta de comunicação na troca de informações entre os componentes, assim como a falta de tratamento para essas falhas. Para os testes de integração de sistemas, há, além dos já citados, os seguintes defeitos típicos: mensagens incoerentes entre os sistemas e não seguimento de especificações obrigatórias de segurança.

Considere o seguinte código:

```
public class SistemaPedidos {  
    private ProdutoDAO produtoDAO;  
    private PedidoDAO pedidoDAO;  
    private EstoqueService estoqueService;  
  
    public void realizaPedido(Pedido pedido) {  
        List<Produto> produtos = produtoDAO.buscaProdutos(pedido.getIdsProdutos());  
        if (!estoqueService.temEstoque(produtos)) {  
            throw new IllegalArgumentException("Produto sem estoque");  
        }  
    }  
}
```

```
}  
estoqueService.diminuiEstoque(produtos);  
pedidoDAO.salvaPedido(pedido);  
}  
}
```

Agora, analise a seguinte situação. A classe **SistemaPedidos** representa um sistema de gerenciamento de pedidos. Ela depende de outras três classes: **ProdutoDAO**, **PedidoDAO** e **EstoqueService**.

Nesse caso, o teste de integração desse sistema verificaria se essas três estruturas estão sendo utilizadas de forma correta pela classe **SistemaPedidos**. Dessa forma, seriam realizados testes para verificar se a busca dos produtos pelo ID está funcionando corretamente. Ao acessar a classe **ProdutoDAO**, verifica se o estoque está funcionando corretamente; ao acessar a classe **EstoqueService**, verifica se está diminuindo corretamente do estoque também presente na classe **EstoqueService**; por fim, ao acessar a classe **PedidoDAO**, verifica se a persistência do pedido está funcionando de forma correta. Garante-se, assim, que todas essas partes do sistema estejam funcionando corretamente em conjunto.

Teste de sistema

É realizado no momento em que o sistema está mais próximo para a liberação de uso ao usuário final. Com base nos dados gerados nesse teste, é decidido ou não se haverá a liberação para a publicação do sistema, pois, como os testes são realizados utilizando um ambiente próximo ao contexto que o usuário utilizará, falhas impactantes podem mudar o rumo de uma finalização do projeto.

Praticamente todas as funções do sistema são testadas, incluindo requisitos legais e regulamentares do *software*. Por exemplo, em um *software* da área contábil, esse nível de teste verificará se os cálculos de impostos estão adequados ao que está estipulado por lei.

Os objetivos assemelham-se aos níveis anteriores, buscando verificar se o sistema segue os itens projetados e especificados, assim como o ponto de oclusão em que está, encontrando defeitos e evitando que falhas possam chegar ao nível de uso do usuário, ou seja, liberar um produto que tem sérios *bugs* e falhas.

A base de teste são os casos de uso, as *users stories*, os diagramas de estado, os manuais de usuário, o relatório de análise de riscos e as especificações de requisitos do sistema. Com isso, são testadas as aplicações em si, os sistemas operacionais, os sistemas de *hardware* e *software* e as configurações do sistema.

As falhas esperadas nesse nível são baseadas em critérios definidos pelos manuais do sistema, por exemplo, cálculos expressos e situações condicionais que devem ser seguidas conforme o fluxo esperado, além de encontrar falhas nos comportamentos definidos por meio dos requisitos funcionais e não funcionais do sistema.

Teste de aceite

O objetivo deste nível de teste é garantir que o sistema esteja operando conforme as definições, expressando a qualidade do *software* na sua totalidade, de modo que o usuário final não encontre problemas de falhas. Caso *bugs* sejam encontrados nesse ponto, significa que existem grandes riscos ao escopo do *software*, pois o objetivo desse teste é não encontrar falhas, tendo em vista que o nível anterior, o teste de sistema, deve coletar informações essenciais para o aceite ou não da publicação do projeto. Logo, disponibilizar um sistema com problemas pode causar transtornos à área de *marketing* da empresa desenvolvedora, que ficará marcada por permitir o lançamento de um projeto incompleto.

Neste nível de teste, pode ocorrer a integração de uma equipe de colaboradores, operadores e administradores da empresa que utilizará o sistema, de modo que as pessoas que utilizarão o sistema validem o projeto. Essa prática é conhecida como teste de aceite de usuário.

Outra prática é o teste de aceite operacional, em que são executados testes que visam à confiabilidade do sistema em se manter operando de forma adequada para os usuários do ambiente de produção, realizando validações em *backups*, restaurações, *performances* e vulnerabilidades de segurança do sistema. Também é realizada a prática de teste de aceite contratual e regulatório, focando a garantia das definições contratuais que foram estabelecidas no momento em que o acordo foi oficializado via assinatura contratual. Os resultados são supervisionados por possíveis testemunhas ou agências regulamentadoras, durante a execução dos testes.

A última prática deste nível de teste é a de alfa e beta, realizada por organizações que já vendem soluções prontas, ou seja, empresas que seguem um padrão de desenvolvimento e disponibilizam seus projetos de forma padronizada no mercado, conhecidas também como desenvolvedoras de *software* de prateleira.

Essa é uma prática que visa encontrar falhas relacionadas ao ambiente, pois algumas situações não são possíveis de serem validadas pelo time de desenvolvimento. O teste alfa é realizado no ambiente da empresa desenvolvedora, sendo realizado por testadores independentes, clientes ou operadores. Já o teste beta é realizado geralmente por potencial cliente em seu próprio local, utilizando versão do sistema comumente chamado de beta.

A base do nível de teste de aceite considera, de forma geral, basicamente itens da documentação do projeto, sendo estes: casos de uso, requisitos de sistema, documento de regulamentação e normas, contratos legais, procedimentos de instalação e relatório de análise de risco.

Já para aceite operacional, ou seja, quando o sistema está performando como se espera, é realizado tendo como base o pacote de dados, normas e regulamentos de segurança, as métricas de *performance*, os requisitos não funcionais, os procedimentos para recuperação de desastres, os procedimentos de *backup* e a restauração, assim como as documentações de operações e instruções de implantação e instalação.

Os objetos de testes são o próprio sistema, a configuração do sistema e dos dados de configuração, os processos de negócios para um sistema totalmente integrado, os relatórios, os dados de produção existentes e os processos operacionais e de manutenção.

As falhas típicas esperadas são em relação aos requisitos de negócios ou do usuário, à validação das regras de negócio, ao *software* não estar seguindo os requisitos contratuais ou regulatórios, assim como às vulnerabilidades de segurança, às falhas não funcionais e à eficácia da *performance*.

Encerramento

Neste material, você aprendeu quais fatores são essências para a elaboração de uma estratégia de teste, verificou conceitos sobre itens que devem ser considerados e compreendeu o uso de algumas estratégias de abordagem.

Além disso, você também verificou conceitos de níveis de teste, compreendendo que há quatro níveis e que cada nível de teste deve ter um ambiente adequado e as próprias propriedades.



Desenvolvimento de Sistemas

Tipos de teste

Testar um *software* é um processo organizado e planejado. Além disso, não é um processo único, pois tem várias técnicas e características. Com a evolução na engenharia de *software* e do desenvolvimento em si, novos tipos de teste foram sendo pensados de maneira que cubra os mais diversos aspectos de um *software* – desde sua interface e a interação com o usuário até as questões de segurança, passando por questões de desempenho e de garantia da qualidade de *software*.

A seguir, serão explorados vários dos tipos de testes praticados no desenvolvimento de sistema. Vale ressaltar que nem todos os projetos utilizarão todos os tipos de teste; cabe ao time de desenvolvimento e de testes identificar aqueles mais adequados às necessidades do projeto, à realidade da equipe e às exigências do cliente.

Teste manual e automatizado

Os testes de *software* podem ser realizados de duas maneiras: manualmente ou automaticamente, com o apoio de uma ferramenta de automação.

O teste manual é realizado por pessoas, geralmente um analista ou especialista em testes. A pessoa deverá testar passo a passo os casos de teste, com bastante atenção às condições que o teste impõe, e verificar seus resultados. Há um aspecto subjetivo que pode ser positivo no teste manual, já que são seres humanos avaliando o sistema. Essa subjetividade pode revelar outros problemas com baixa usabilidade ou falhas não observadas anteriormente no sistema.

Geralmente é um teste de baixo custo e que não depende de grandes configurações anteriores. Por outro lado, são testes mais lentos e, pela mesma subjetividade que pode ajudar, há boa chance de algum problema passar despercebido por quem testa.



Figura 1 – Benefícios do teste manual

Fonte: adaptado de Global App Testing (c2023)

Os testes automatizados, por sua vez, são realizados por *softwares* ou bibliotecas de linguagem que exigem uma configuração inicial (*scripts*, códigos ou passos configurados) para a execução de um teste. Após essa configuração, o teste pode ser executado e reexecutado com muita agilidade e sem a necessidade da interferência de uma pessoa, mostrando, na ferramenta, se o resultado esperado para uma funcionalidade testada foi alcançado ou não.

Esses testes são bastante lógicos: se o teste produzir determinado valor ou atingir resultado esperado, então ele passou; se o teste produzir valor ou resultado divergente do esperado, então ele falhou. Por isso, são muito confiáveis, sem risco de interferência em razão

de subjetividades. Podem ser mais caros, pois dependem de ferramentas específicas, e não detectarão problemas que pessoas poderiam observar, como de usabilidade. Também levam mais tempo (e exigem mais *expertise*) para serem configurados ou programados, mas, uma vez que isso é feito, a execução dos testes é muito mais veloz que o teste manual.



Figura 2 – Benefícios do teste automatizado

Fonte: adaptado de Testinium (2021)

É comum (e desejável) que ambos os tipos de teste sejam utilizados no processo de teste de um *software*. Algumas funcionalidades e alguns aspectos do sistema se beneficiarão mais de testes manuais (como nos testes de aceitação, por exemplo, como você verá a seguir), outras são mais beneficiadas com testes automatizados (em testes de regressão, por exemplo). Também se deve levar em consideração a *expertise* da equipe e o tempo disponível para testes. No entanto, em projetos modernos, é recomendável que ao menos um tipo de teste automatizado seja aplicado ao projeto.

Teste funcional

Testes funcionais são aqueles que verificam a funcionalidade e as características de um sistema sem adentrar em questões técnicas, como código-fonte ou estruturas de banco de dados. Por essa razão, também é denominado teste de caixa-preta (em que se vê o exterior do

programa, mas não o interior). O foco nesse tipo de teste fica nos valores informados como entrada de dados e saídas obtidas pelo programa, comparando-as com o que se esperava obter.

Dentro do teste funcional ainda há técnicas próprias que podem ser aplicadas. Mais detalhes podem ser obtidos no conteúdo **Técnicas de teste** desta unidade curricular.

Teste de regressão

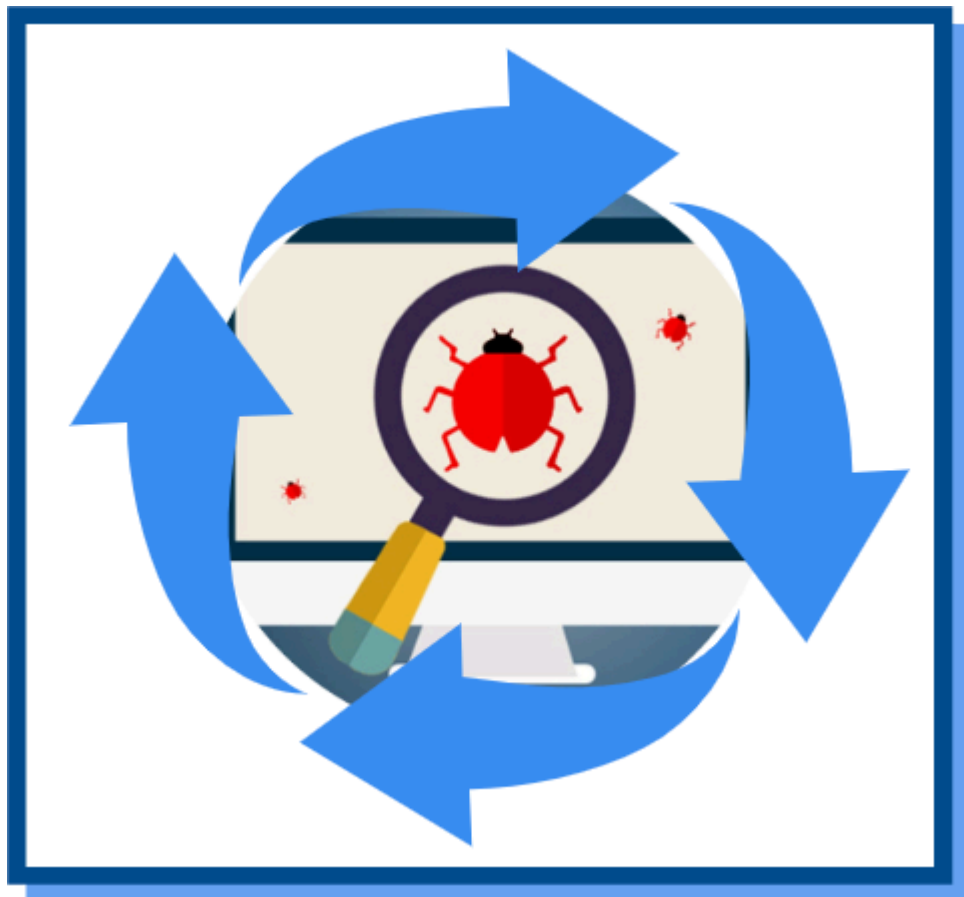


Figura 3 – Testes de regressão

Fonte: Pixabay (2015)

Alterações em código-fonte, correções aplicadas ou funcionalidades novas implementadas em um sistema podem trazer efeitos colaterais no *software*, trazendo falhas ao que já estava funcional anteriormente. Testes de regressão são usados para garantir que alterações em um *software* não afetem negativamente o que já foi testado e está funcionando bem. Procura, portanto, assegurar que o sistema não regrediu em questão de qualidade, mantendo-o confiável.

A forma mais simples de teste de regressão é reexecutar uma bateria de testes manuais previstas nos planos de teste do projeto. Entretanto, em razão de restrições de projeto, de equipe, tempo e ferramentas, pode ser que alguns cenários acabem ficando de fora, o que

pode trazer riscos ao projeto. Isso porque quanto mais o *software* se expande, mais casos de teste vão surgindo, tornando o processo de teste mais oneroso.

Outra alternativa, em muitos casos mais eficiente, é a automatização dos testes, ou seja, usar *softwares* ou bibliotecas de linguagem de programação que permitam a criação de casos de teste e a execução automática desses casos dentro do *software*. Entre essas ferramentas está o teste unitário (em Java, utiliza-se JUnit para isso). Uma vez configurada a ferramenta e desenvolvidos os casos de teste, basta acionar sempre que houver uma mudança no código ou agendar a execução periódica dos testes e aguardar os resultados para verificar se houve alguma falha. Esse tema será abordado com mais detalhe no conteúdo **Testes unitários automatizados** desta unidade curricular.

De forma geral, o mais adequado é que alguns testes sigam sendo realizados manualmente enquanto outros são automatizados.

Passos para realizar testes de regressão

1. Coletar casos de testes que deverão ser reexecutados. Incluir casos de teste que cubram áreas mais propensas a erros no sistema, que sejam mais vulneráveis a mudanças, que testem funções principais do sistema e que sejam mais complexos, como sequências de eventos em interface visual.
2. Estimar o tempo de execução dos testes. É importante levar em consideração os critérios, o plano e a revisão dos casos de teste para calcular o tempo necessário para testes de cada funcionalidade escolhida.
3. Separar os testes que podem ser automatizados. A tendência pode ser de automatizar a maioria dos testes, mas algumas verificações mais complexas, como séries de ações encadeadas em interface visual, podem exigir testes manuais.
4. Priorizar os casos de teste. Recomenda-se que a prioridade sejam funções básicas e centrais do sistema; em seguida, as funções cruciais, mas não centrais, ao *software*; por fim, funções periféricas, mas que possam evitar problemas técnicos e complicações ao desenvolvimento.
5. Aplicar ferramentas para acelerar o processo de teste.

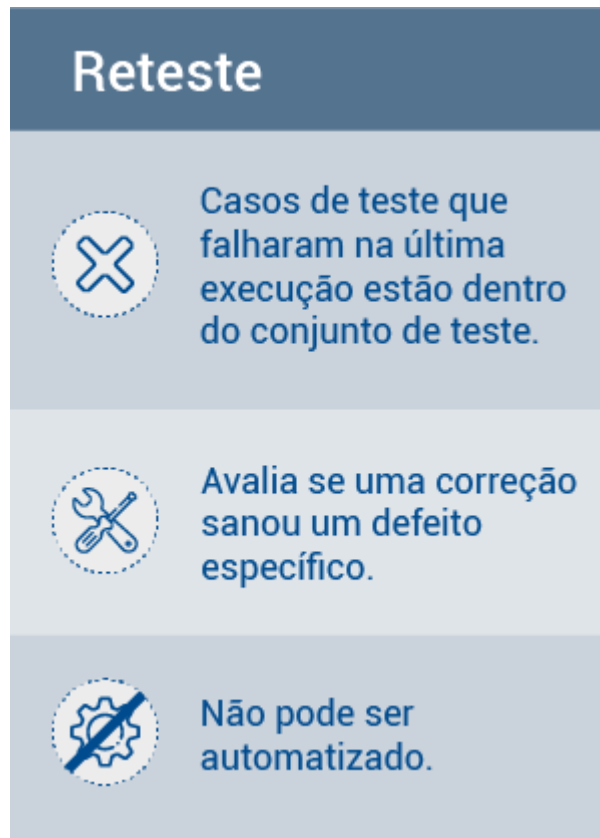
Teste de confirmação



O teste de confirmação é aquele realizado após a correção de um problema verificado no sistema, possibilitando ao testador confirmar que o problema já não ocorre mais ou para retornar ao time de desenvolvimento caso o problema persista. Também é conhecido como reteste e ocorre regularmente nos *sprints* do projeto usando método ágil.

Não há uma técnica específica. Aplica-se, aqui, teste manual ou automatizado. O único ponto é que o testador precisa executar exatamente o mesmo caso de teste que evidenciou o erro (não é necessário criar um novo caso de teste).

Teste de confirmação não é igual a teste de regressão. Enquanto os testes de confirmação são aplicados especificamente a defeitos recém corrigidos, os testes de regressão são mais abrangentes e acontecem após testes de confirmação.



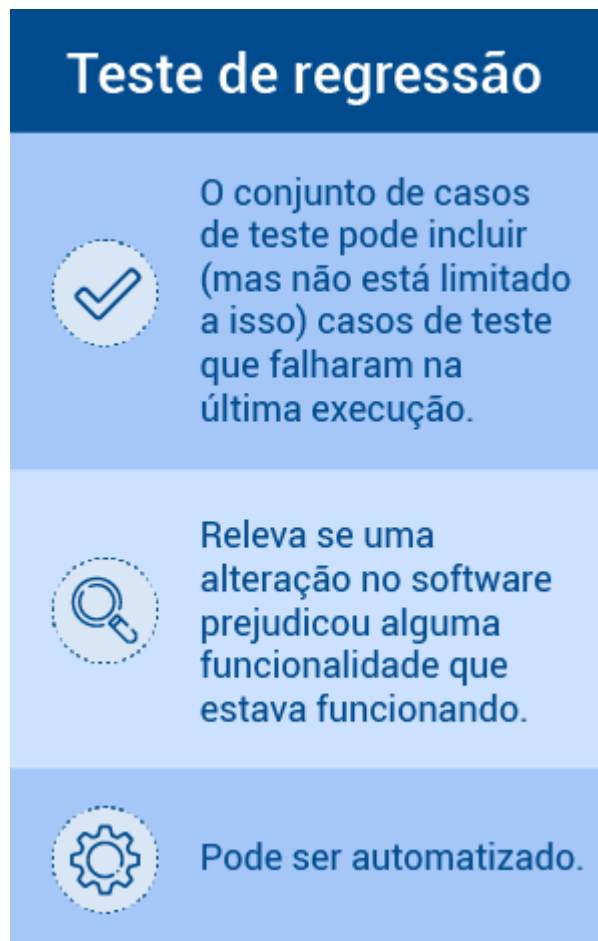


Figura 4 – Reteste (teste de confirmação) *versus* teste de regressão

Fonte: adaptado de Grimms (c2023)

Teste de aceitação

Conceitualmente próximo dos testes funcionais, os testes de aceitação estão mais ligados aos clientes e envolvem os usuários finais do *software* para a validação das funcionalidades e dos requisitos implementados no sistema.



Figura 5 – Teste de aceitação certifica o que está de acordo ou não com o que o usuário final necessita

Fonte: Pixabay (2014)

Geralmente são testes elaborados com base em histórias do usuário. O teste de aceitação, que parte da visão do cliente sobre o sistema, diferencia-se, assim, dos demais testes de sistema, que partem da visão dos desenvolvedores sobre os requisitos.

Em metodologias mais antigas de desenvolvimento *software*, geralmente seriam testes realizados apenas no final do desenvolvimento e antes da implantação. Com os métodos ágeis e uma proximidade maior do cliente, geralmente acontecem em ciclos (no Scrum, testes de aceitação podem ser realizados nas reuniões de revisão dos *sprints*).

Há três estratégias principais para teste de aceitação. Veja, a seguir, quais são elas.

Testes formais

São elaborados com base em planos de testes pensados com o mesmo detalhamento dos testes de sistema. Podem ser automatizados, executados pelo time de teste ou executados por representantes do cliente.

Testes informais ou testes alfa

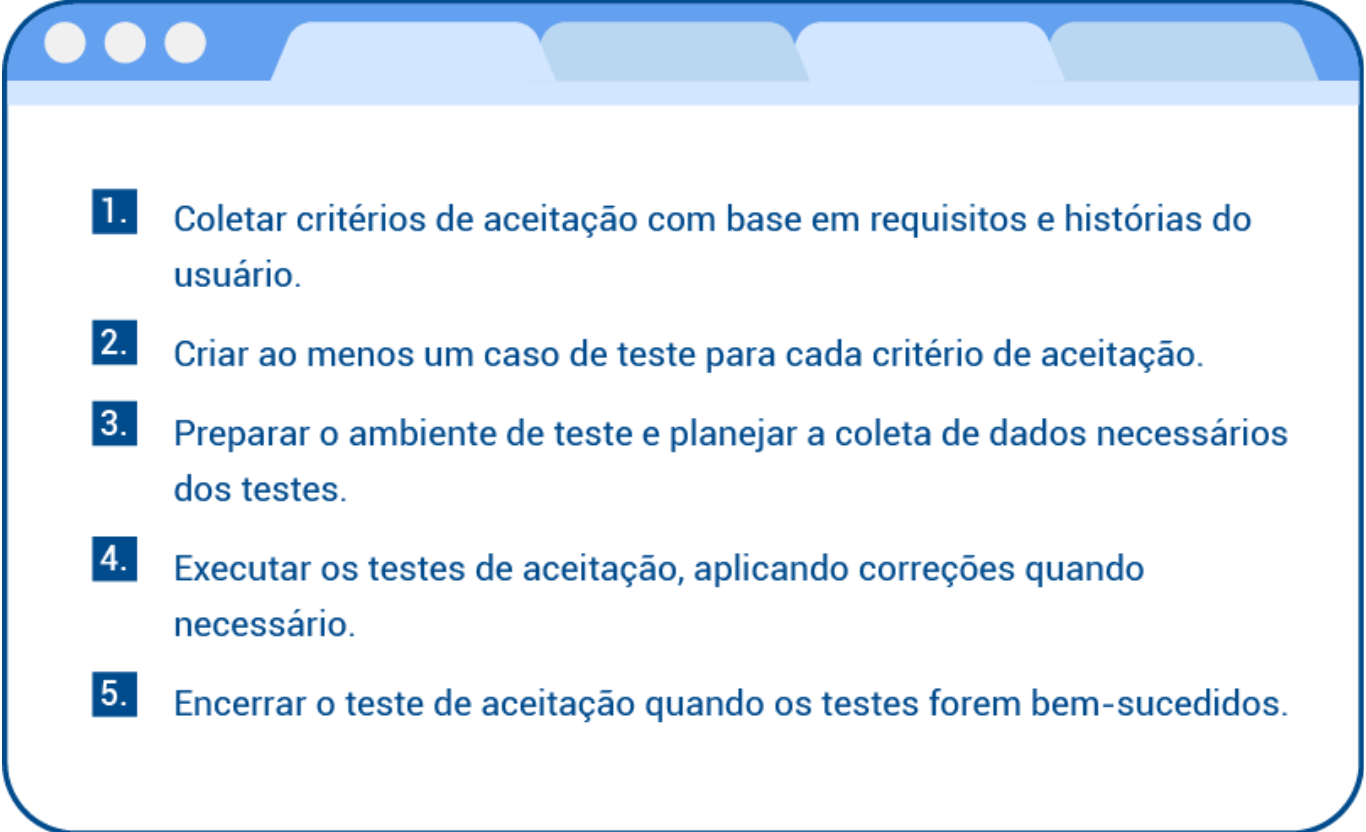
Geralmente são executados pelo cliente. São testes sem um roteiro definido (as funcionalidades do sistema a serem exploradas são identificadas e documentadas, mas não são desenvolvidos casos de testes para serem seguidos). Os usuários testadores exploram o sistema, executando tarefas cotidianas e observando inconsistências.

Testes beta

Nesta modalidade, um número maior de potenciais usuários (não necessariamente ligados ao cliente) fazem testes livremente. É um tipo de teste bastante aplicado em *softwares* como o WhatsApp, que disponibiliza versões exclusivas do aplicativo com novas funcionalidades a usuários cadastrados. Com base nos dados coletados do uso desses testadores, a equipe pode ajustar ou remover funcionalidades do sistema.

Como realizar teste de aceitação

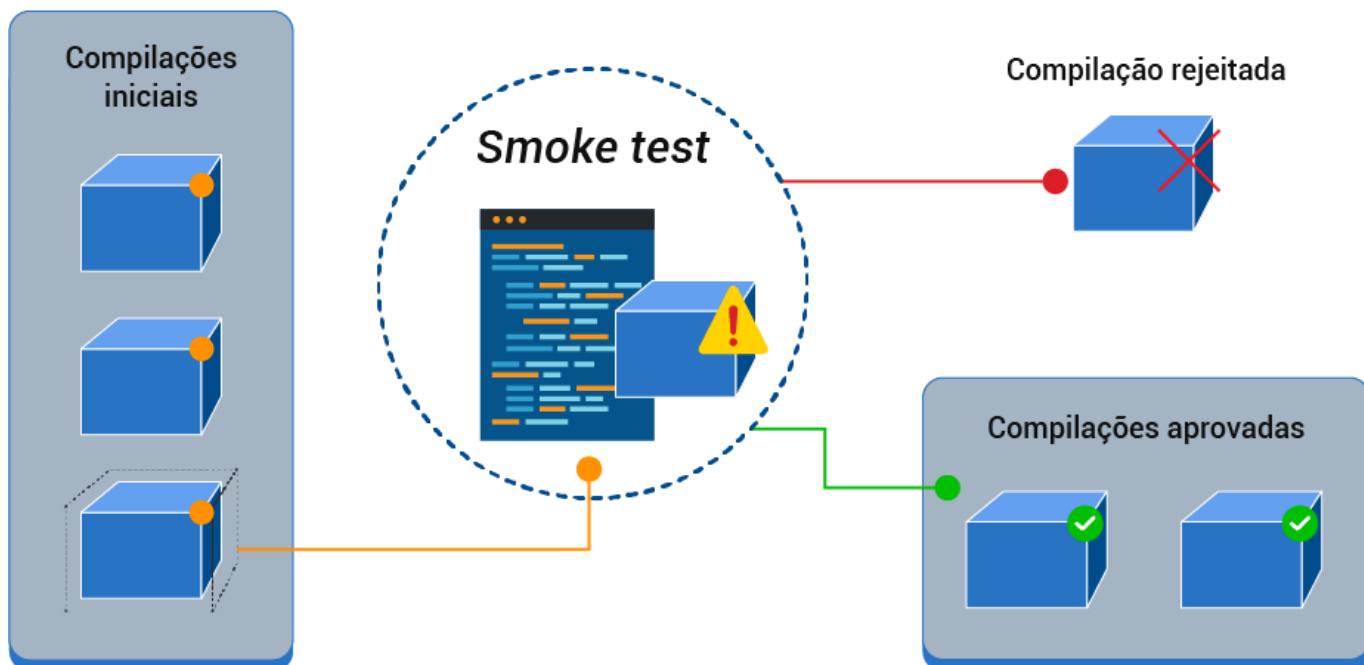
Os passos gerais para realizar atividades de teste de aceitação são:

- 
1. Coletar critérios de aceitação com base em requisitos e histórias do usuário.
 2. Criar ao menos um caso de teste para cada critério de aceitação.
 3. Preparar o ambiente de teste e planejar a coleta de dados necessários dos testes.
 4. Executar os testes de aceitação, aplicando correções quando necessário.
 5. Encerrar o teste de aceitação quando os testes forem bem-sucedidos.

Smoke test

Os *smoke tests*, ou testes de fumaça, são testes rápidos que verificam as funcionalidades básicas de um sistema. Esse teste deve determinar se o *build* (a compilação) do sistema está gerando um *software* estável ou não. É semelhante a um teste de aceitação, mas de maneira bastante resumida, devendo se ater aos recursos mais importantes da aplicação, ou seja, é um teste mínimo executado rapidamente após um *build*.

Sempre que novas funcionalidades são desenvolvidas e integradas ao sistema, recomenda-se a aplicação do *smoke test*. O objetivo é informar problemas com agilidade e bloquear a versão, caso necessário.

Figura 6 – Fluxo do *smoke test*

Fonte: adaptado de Martins (2021)

Os testes podem ser manuais ou automatizados. No caso do teste manual, o time de testadores recebe a nova versão compilada do *software* e executa os casos de teste de alta prioridade. Se eles falharem, a versão é rejeitada e retorna ao time de desenvolvimento para correções. Se os testes passarem, os demais testes funcionais são executados de acordo com o previsto no projeto.

Os testes automatizados podem agilizar bastante o *smoke test*, podendo executar imediatamente antes ou depois da compilação do sistema. Os desenvolvedores podem rapidamente reter a versão para correções, não precisando passar pelo time de testadores.

Teste exploratório

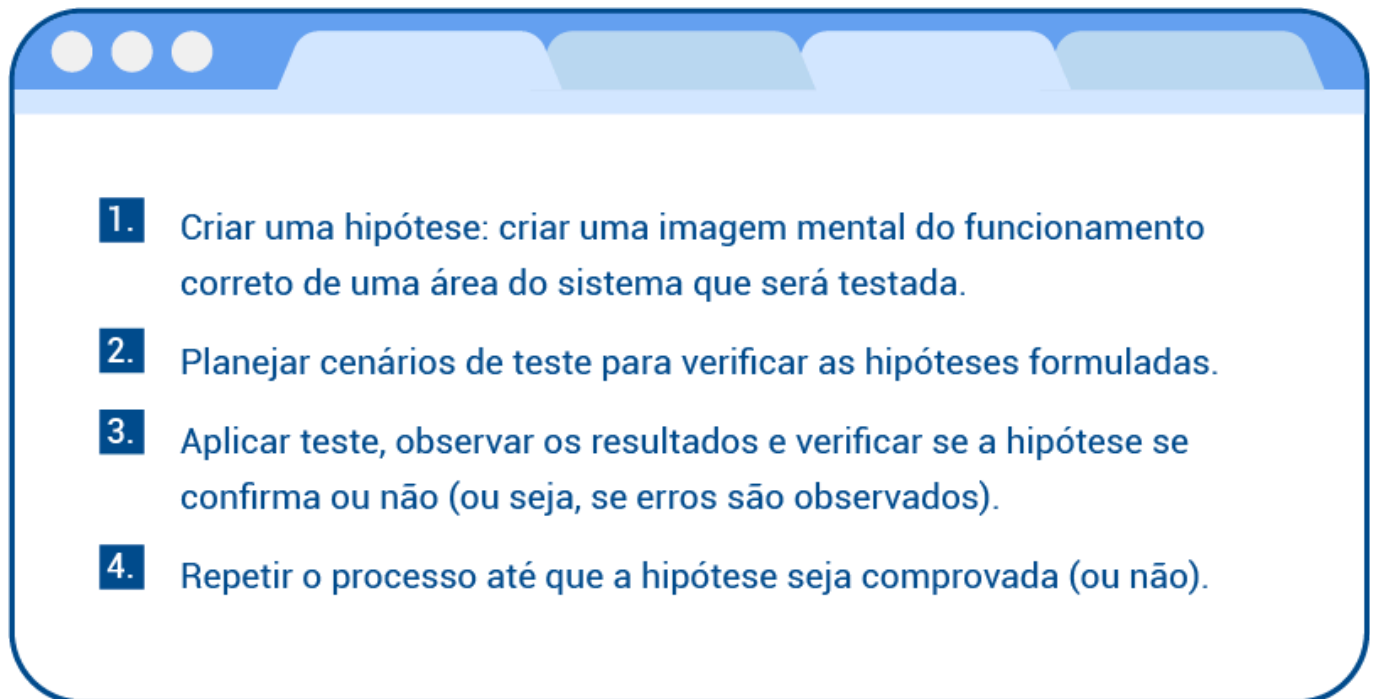
Por definição, nos testes exploratórios, a criação e a execução dos testes acontecem ao mesmo tempo. Ou seja, normalmente são testes que não seguem uma definição formal de planos de testes, mas, sim, a experiência e as observações do testador. Daí o nome do teste: o testador responsável fará uma exploração mais ou menos livre pelo *software* em busca de inconsistências.

Esse tipo de teste pode ser interessante para casos em que não se tem um conhecimento prévio sobre o sistema; em que não há requisitos formais documentados; em que há pouco tempo disponível; em que seja necessário identificar passos de um defeito aleatório do sistema,

em projetos que aplicam metodologias ágeis, que dispensam formalidades e documentações. Também pode ser aplicado como apoio à definição formal de casos de teste no sistema, podendo auxiliar a criação ou expansão de casos de teste do sistema.

Apesar de ser um processo com mais liberdade, a exploração não acontece de maneira aleatória. Geralmente, esses testes iniciam pelo questionamento sobre qual é o teste mais importante que se deve realizar no momento e o planejamento do teste desdobra-se disso.

Uma proposição para o processo de teste exploratório é a seguinte:



Teste de desempenho

Um teste geralmente relegado ao final do processo de desenvolvimento de *software*, mas que deveria estar presente desde o início do projeto, é o de desempenho, uma modalidade que avalia tempo de processamento, velocidade de transferência de dados, eficácia de consulta em banco de dados, número máximo de usuários conectados, uso de processador e memória da máquina, uso de rede, tempo de resposta a comandos do usuário, entre outros aspectos não funcionais do sistema de *software*.



Figura 7 – Testes de *performance* verificam a eficiência do programa

Fonte: Okeke (2022)

Veja algumas razões para realizar testes de desempenho:

- ◆ Verificar se a aplicação satisfaz requisitos não funcionais definidos no projeto.
- ◆ Encontrar gargalos no sistema, ou seja, momentos específicos em que o sistema demora mais que o normal para responder.
- ◆ Verificar a estabilidade do sistema sob pico de acessos.

No contexto de desenvolvimento ágil, o teste de desempenho pode ser aplicado regularmente a cada *sprint* ou em momentos adequados para isso.

O processo de teste de desempenho variará bastante de empresa para empresa e de projeto para projeto. Algumas técnicas como teste de estresse, carga e volume, abordadas a seguir, podem fazer parte do processo caso a intenção seja testar, por exemplo, a resiliência de um sistema a acessos simultâneos.

De maneira geral, recomenda-se que primeiro sejam identificados os indicadores não funcionais a serem testados. Com base nisso, descobrem-se quais são as ferramentas adequadas para esse teste (*softwares* de simulação de acesso, por exemplo, poderiam ser usados para verificar número de usuários máximo executando o sistema ao mesmo tempo). Também é importante definir o ambiente (rede, banco de dados, condições de Internet que sejam próximas do cliente, entre outros) onde os testes executarão.

Definem-se, então, os critérios de aceite para os testes (por exemplo, qual é um bom número de usuários conectados simultaneamente) e, com base nisso, estabelecem-se os planos de teste. Com as ferramentas preparadas, então, executam-se os testes, verificando os problemas e realizando os ajustes necessários. É importante que os testes sejam executados repetidas vezes, para confirmar os problemas de desempenho.

Teste de carga

Esse é um tipo de teste não funcional especialmente interessante para aplicações *web*. O teste de carga estabelece e avalia a capacidade de processamento de um *software* dada uma carga crescente de dados, geralmente conexões de usuários.

Em uma aplicação *web*, o teste de carga pode ser avaliado simulando um número X de conexões inicialmente e verificando a disponibilidade do sistema. Depois, aumenta-se esse número de conexões, revendo como o sistema está operando, e assim sucessivamente até que se alcance um número próximo ao que seria alcançado no dia a dia do sistema ou até que o sistema apresente falha.

Analogamente, seria possível verificar carga de processos de banco de dados, aumentando gradativamente o número de consultas geradas pelo sistema e verificando a estabilidade de sua conexão com o banco de dados.

Com base nos testes de carga, é possível estabelecer a capacidade da aplicação e identificar fatores limitantes, que podem ser de *hardware*, de codificação, de rede, entre outros.

Os resultados desse teste geralmente são representados pelo número de transações simultâneas e pela quantidade de usuários conectados ao mesmo tempo no sistema. Podem indicar problemas não só em código como em infraestrutura (a necessidade de um servidor com maior capacidade de processamento, por exemplo, ou uma conexão à Internet com maior banda).

Teste de estresse



Enquanto os testes de carga geralmente simulam uma alta demanda cotidiana de um sistema, o teste de estresse (ou teste de esforço) simula uma demanda extraordinária, que poderá acontecer pontualmente, mas que poderá levar à falha do sistema. Esses testes são, portanto, usados para garantir a estabilidade e a confiabilidade de um sistema, simulando uma alta taxa de tráfego de dados e registrando a resposta do *software* a essa demanda. É um teste indicado, por exemplo, para sistemas *web* que, em razão de uma data específica ou um evento, precisam testar seus limites.

Como exemplo, é possível citar um *site* de vendas em uma data de liquidação ou um *site* de inscrições limitadas para um evento. Ambos os casos apresentarão, em um período, uma situação de acesso muito acima do que acontece diariamente. Assim, um teste de estresse pode prevenir que o sistema caia por excesso de acessos.

Além de servir para verificar quanto o sistema suporta de tráfego de dados, o teste de estresse também serve para avaliar como o sistema se comporta após uma falha.

Com base no teste de estresse, assim como no teste de carga, é possível definir ajustes em infraestrutura e no *software* para garantir mais robustez ao sistema.

Algumas métricas podem ser avaliadas baseando-se no teste de estresse:

- ◆ Quantidade de páginas requisitadas por segundo (em sistemas *web*)
- ◆ Taxas de transferência (tamanho dos dados de resposta em sistema *web*)
- ◆ Tempo de carregamento de uma tela ou página
- ◆ Número de falhas na conexão

Uma das ferramentas mais usados para testes de carga e de estresse é o JMeter, em que o desenvolvedor pode configurar cenários de teste e o *software* fará simulação de acessos ao sistema.

Teste de volume

Mais um teste de desempenho, o teste de volume verifica o comportamento do sistema em situações de grande carga de dados. Diferente dos testes de carga e de estresse, que se preocupam essencialmente com o tráfego de informações e conexões estabelecidas, o teste de

volume trabalha com tamanho de arquivos e bancos de dados usados pelo sistema testado.



Para o teste, estende-se o tamanho do banco de dados ou arquivo até um limite estabelecido e, então, o desempenho do sistema é testado, observando-se o tempo de resposta para as ações do usuário e o comportamento do sistema (se erros acontecem, se as funcionalidades são executadas corretamente).

O teste de volume é interessante porque todo sistema, com o passar do tempo, recebe quantidades de dados que podem diminuir o desempenho do *software*. No tempo de desenvolvimento, os testes funcionais geralmente acontecem sobre pequenos volumes de dados. Assim, o teste de volume pode detectar problemas não observados nos testes comuns. Os dados gerados para o teste geralmente vêm de um *software* desenvolvido para povoar com dados válidos o banco ou os arquivos usados pelo sistema.

Com base no teste de volume, pode-se identificar, então, o ponto em que a estabilidade do sistema diminui e a capacidade de dados da aplicação. Alguns dados podem ser levados em consideração na análise desses testes:

- ◆ Tempo de resposta do sistema (o sistema demora mais, ou não, quando está sob grande volume de dados)
- ◆ Perda de dados (o sistema passa a perder informações quando se trabalha com banco de dados volumosos)
- ◆ Armazenamento e sobrescrita de dados (os dados continuam sendo gravados corretamente, ou não, com um banco de dados muito povoado)

Entre as vantagens do teste de volume, estão a capacidade de detectar melhorias necessárias em infraestrutura e em código para o sistema, a identificação precoce de gargalos de desempenho e a garantia de que o sistema poderá ser usado no mundo real.

Teste de recuperação

É denominado teste de recuperação o processo de teste não funcional realizado para verificar a resiliência do sistema, se ele consegue se recuperar de falhas ou não. No teste, força-se a ocorrência de uma falha e verifica-se como o sistema se comporta a seguir.

É importante que o sistema seja desenvolvido de tal maneira que, após uma falha, ele possa continuar sua execução tratando essa falha e retornando a um estado consistente anterior do sistema. Essa recuperação deve acontecer em um tempo específico e é essencial

em sistemas críticos como de defesa, dispositivos médicos e outros.



Como exemplo de recuperação, pode-se imaginar uma aplicação baseada em rede, processando envios e recebimentos de dados. Se desconectar o cabo de rede e algum tempo depois restabelecer a conexão, então o *software* deve ser capaz de retomar a transmissão de dados do exato ponto onde parou quando perdeu conexão.

Algumas situações previsíveis podem ser testadas para recuperação:

- ◆ Falha na alimentação de energia na máquina onde o programa está executando (ao retornar, terá salvado as últimas alterações?)
- ◆ O servidor em uma aplicação de cliente-servidor não está acessível (o programa no cliente permitirá executar outras ações localmente?)
- ◆ Dispositivo externo não está respondendo (o sistema trava por causa disso?)

É importante que o testador se prepare para o processo de teste de recuperação:

- ◆ Assegurando-se do ambiente de execução do *software* e da capacidade de expansão (processador, memória etc.)
- ◆ Estudando o impacto e a gravidade de possíveis falhas
- ◆ Observando que o plano de teste contemple testes de recuperação
- ◆ Realizando a manutenção de *backups* do sistema com vários estados do *software* e do banco de dados, para o caso de a falha ser severa a tal ponto que se percam dados
- ◆ Documentando todas as etapas do teste de recuperação

Quando se observa mais de uma falha, ao invés de se ocupar com todas as falhas de uma vez, os testadores devem estruturar a situação, escolhendo e priorizando um segmento de cada vez.

Como vantagens do teste de recuperação, é possível citar: a melhoria geral na qualidade do sistema, a elaboração de planos de recuperação de desastres para o sistema, a eliminação de riscos e o aprimoramento do desempenho geral do sistema, tornando-o mais confiável.

O processo também conta com desvantagens: pode ser demorado e custoso por envolver várias etapas e preparações antes e durante os testes; precisa contar com profissionais treinados e capazes de observar de maneira abrangente as falhas e a recuperação; pode apresentar algumas falhas imprevisíveis ou difíceis de alcançar.

Teste de segurança

Se um sistema armazena dados sensíveis sobre seus usuários ou sobre dados de negócio do cliente, então a segurança desses dados é um requisito, pois é um potencial alvo de acesso impróprio ou ilegal. Para isso, é importante que o sistema tenha um processo de testes de segurança, que buscam brechas ou vulnerabilidades que podem ser exploradas por *hackers* que tentam invadir o sistema e acessar e/ou corromper dados sensíveis.

O objetivo dos testes de segurança é, portanto, identificar todas as lacunas e as fraquezas de um sistema que possam resultar em perda de informações, prejuízos financeiros e prejuízos na reputação do cliente ou das pessoas envolvidas por dados que param em mãos de terceiros. Tais problemas podem ser solucionados com ajustes no código do sistema ou na infraestrutura.

Vale observar que são testes aplicados para aplicações que estejam conectadas a uma rede (seja interna, seja externa, seja Internet). É um tipo de teste especialmente importante para sistemas *web*, mas que pode (e deve) ser aplicado para sistemas que funcionam em rede local.

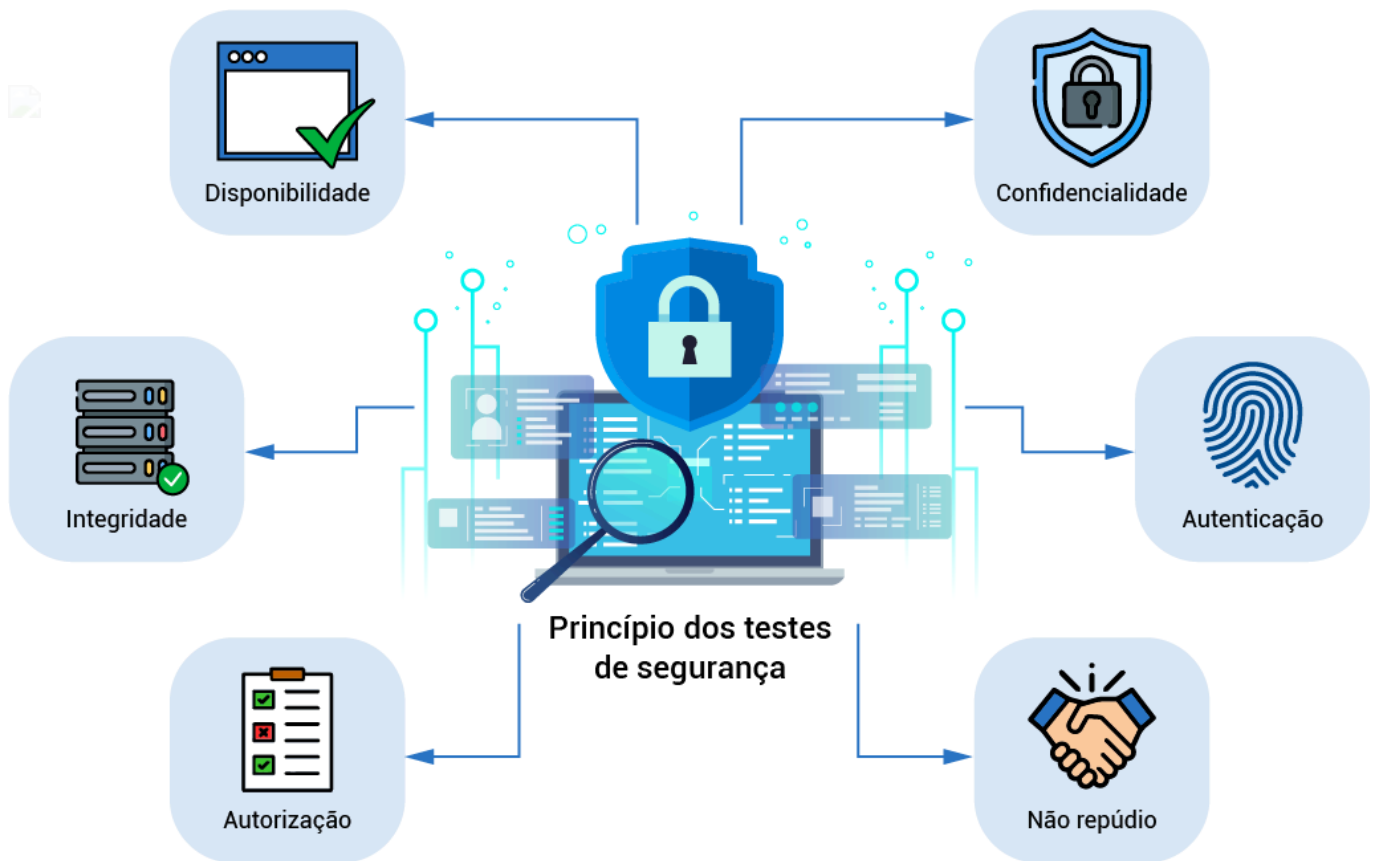


Figura 8 – Princípios dos testes de segurança

Fonte: Senac EAD (2023)

É possível classificar sete tipos de teste de segurança. Veja, a seguir, quais são eles.

Varredura de vulnerabilidade

Um teste automatizado em que um *software* varre o sistema buscando vulnerabilidades conhecidas. Alguns exemplos de ferramentas usadas são o SecPod SanerNow, o Inviecti (ambos pagos) e o Nikto2 (gratuito).

Varredura de segurança

Pode ser manual ou automatizada e tem por objetivo buscar pontos fracos na rede e no sistema, produzindo soluções a esses riscos.

Teste de penetração

Busca por vulnerabilidade de acesso externo, simulando ataques *hackers* no sistema. Também é conhecido como *pentest*.

Avaliação de risco

É uma análise sobre os riscos de segurança observados na organização, classificando-os como baixo, médio ou alto e prevendo medidas para reduzi-los.

Auditoria de segurança

É uma inspeção interna dos aplicativos, dos sistemas operacionais e das linhas de código do sistema em busca de falhas de segurança.

Hacker ético

Com a ajuda de um hacker contratado para esses testes, realizam-se tentativas sucessivas de invasão ao sistema, expondo falhas de segurança. Também é conhecido como teste *red team*.

Avaliação de postura

É uma classificação da postura geral da empresa com relação à segurança obtida baseando-se na combinação da varredura de segurança, no hackeamento ético e nas avaliações de risco.

Outra classificação se dá de acordo com o momento em que o teste de segurança ocorre e com a origem das ações. A seguir, conheça o SAST (*static application security test*), o DAST (*dynamic application security testing*) e o IAST (*interactive application security testing*).

SAST

Testes de segurança de aplicativo estático, que acontecem nas etapas de desenvolvimento e, ao encontrarem falhas e vulnerabilidades, passam rapidamente para correção. São testes internos, realizados com base no próprio ambiente de desenvolvimento.

DAST

Testes de segurança de aplicativo dinâmico, que são realizados antes da liberação do aplicativo ao cliente (seja a entrega de um sistema *desktop*, seja a disponibilização de um sistema *web* ao público). São realizados da parte externa para a interna, ou seja, testa-se em um ambiente externo ao de desenvolvimento, possibilitando a identificação de vulnerabilidades mais visíveis. Também analisam questões de desempenho, como tempo de execução.

IAST

Testes de segurança de aplicativo dinâmico, que são internos e concedem ao testador ou à ferramenta automatizada de teste um acesso maior aos dados e ao código, permitindo detecções mais precisas de vulnerabilidades. Geralmente acontecem no início do ciclo de vida do projeto.

Veja, a seguir, as técnicas que podem ser aplicadas no processo de teste.

Acessos à aplicação

Testar as permissões dos diferentes usuários do sistema para verificar se algum perfil está acessando indevidamente recursos que deveriam estar bloqueados a ele. Por exemplo: um vendedor não deve acessar os dados de outros vendedores e informações financeiras em um sistema para uma loja. Para cada permissão, o testador deverá verificar todos os menus, as telas e as funções para detectar se algum recurso está indevidamente acessível.

Proteção de dados

Além de recursos do sistema, o acesso a determinados dados pode ser restrito a alguns usuários. No exemplo anterior, os dados de salários e faturamento da loja não devem estar ao alcance dos vendedores. Isso pode ser resolvido por meio de permissões no sistema,

possivelmente em nível de banco de dados, mas também com encriptação de dados (para senhas e dados de cartões de crédito, por exemplo).

Ataques de força bruta

Usar um *software* para, com base em um usuário conhecido, testar senhas uma após a outra até conseguir acesso ao sistema. Geralmente sistemas aplicam uma restrição de quantidade de tentativas de autenticação, inibindo esse tipo de ataque (ao tentar acessar uma rede social, por exemplo, após um número de tentativas malsucedidas, o acesso pode ficar bloqueado por algumas horas e um *captcha* ou uma confirmação de identidade por outro meio pode ser requisitada).

SQL Injection e XSS (*cross-site scripting*)

Um *script* é usado maliciosamente para tentar invadir ou prejudicar dados do sistema. No caso de SQL Injection, aproveitam-se as vulnerabilidades na implementação de banco de dados, como consultas não protegidas, e usam-se comandos SQL para a invasão. Já o XSS trata de códigos de *script* de uma página que apontem para outra página, com a intenção de obter dados ou acesso a esta.

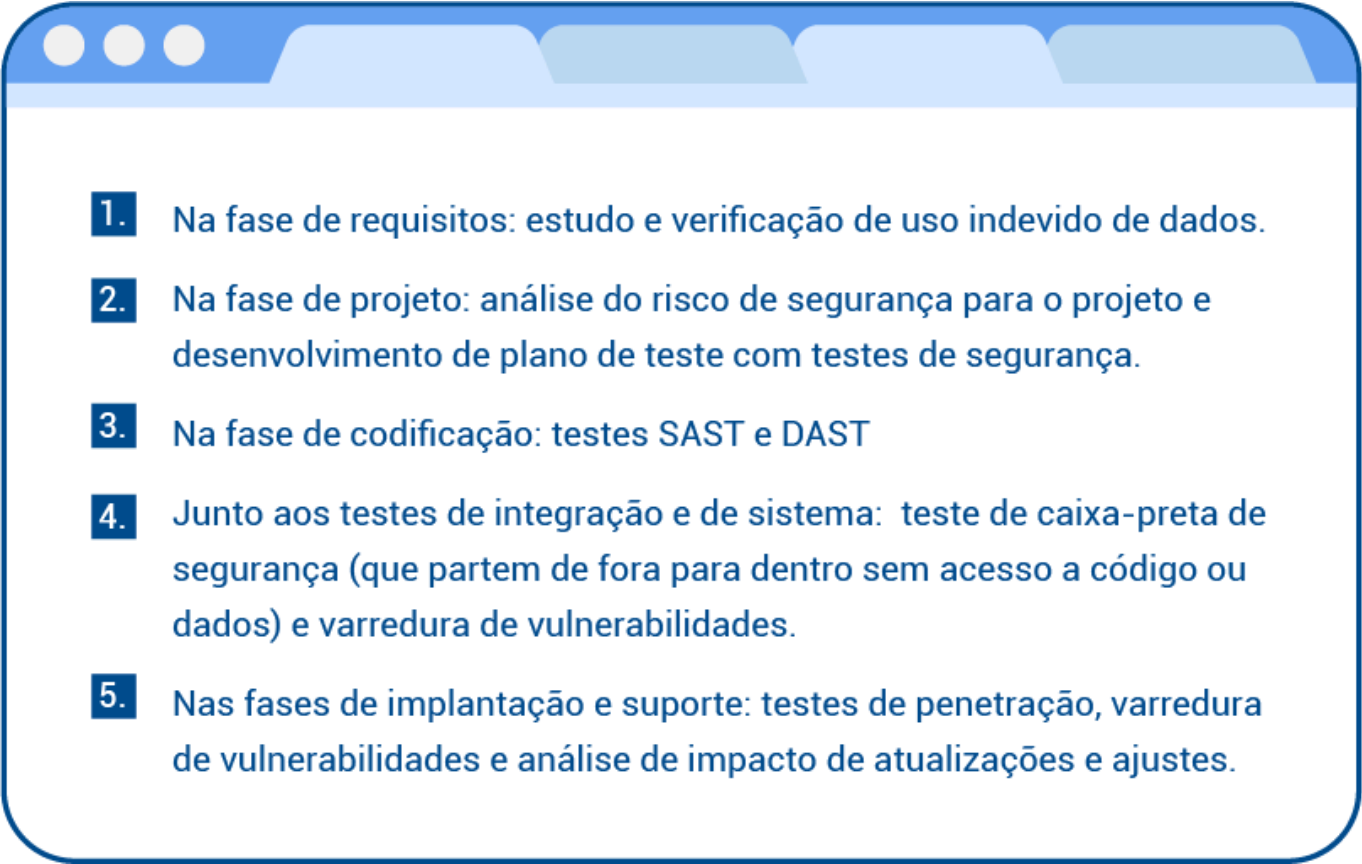
Pontos de acesso a serviços

Aplicações *web* atuais muitas vezes expõem serviços para que outras aplicações possam acessar. Nesse contexto, é importante testar se apenas as aplicações autorizadas estão tendo acesso aos serviços autorizados e se, com base nesse serviço, não se obtém acesso a outras funcionalidades ou dados que deveriam estar protegidos.

Manejo de erros

É importante informar ao usuário quando um erro acontece, mas nunca devem aparecer dados detalhados da falha, que podem explicitar detalhes do sistema e podem ser usados por invasores para obter acesso. Assim, uma exceção (e principalmente seu *stack-trace*) não deve ser exposta ao usuário. Ao invés disso, esses detalhes podem ser registrados em arquivo interno de *log*, que deve estar protegido de acesso externo, mas acessível aos desenvolvedores para investigar problemas.

Considerando o processo tradicional de desenvolvimento de *software*, de maneira geral pode-se dizer que os testes de segurança devem percorrer todo o processo de desenvolvimento do sistema.

- 
1. Na fase de requisitos: estudo e verificação de uso indevido de dados.
 2. Na fase de projeto: análise do risco de segurança para o projeto e desenvolvimento de plano de teste com testes de segurança.
 3. Na fase de codificação: testes SAST e DAST
 4. Junto aos testes de integração e de sistema: teste de caixa-preta de segurança (que partem de fora para dentro sem acesso a código ou dados) e varredura de vulnerabilidades.
 5. Nas fases de implantação e suporte: testes de penetração, varredura de vulnerabilidades e análise de impacto de atualizações e ajustes.

No caso de métodos ágeis, os testes de segurança deverão estar presentes em todos os *sprints*, em seu planejamento, na implementação e na finalização do incremento de *software*.



Desenvolvimento de Sistemas

Plano de teste

Introdução

Uma etapa essencial em todas as fases de criação de um *software* é o planejamento, ou seja, especificar como ocorrerão todas essas fases, desde equipe, tempo de duração do projeto, cronograma de cada etapa, quais diagramas serão executados, que dados serão armazenados no banco, como será a interface etc. Certamente, todo programador e toda equipe de desenvolvimento esperam ansiosamente ver o projeto funcionando, porém, para que o sistema seja bem-sucedido, a etapa do planejamento não pode ser burlada.

O teste de um sistema não pode ser feito de qualquer maneira, apenas colocando dados depois do *software* pronto e vendo se ele “roda”. Existem diversos critérios e muitas particularidades que precisam ser levados em consideração, e é neste momento que entra um plano de teste, que é um documento que visa planejar as atividades de teste durante todo o projeto e os métodos de testagem que serão empregados. Ainda, deve estabelecer as métricas do que é considerado aceitável ou não como resultado de um teste, as respostas da equipe perante erros que possam aparecer, além de diversos outros elementos que serão ainda apresentados.

Especificações e tipos de teste

O plano de testes durará durante toda a vida útil do projeto de *software* que está sendo criado. Normalmente é elaborado por um gerente de testes, ou pelas pessoas mais experientes da equipe, com a ajuda dos demais envolvidos no desenvolvimento, podendo ter a participação de pessoas de outras áreas que tenham relevância para o documento, por exemplo, *designers*.

Os planos de testes nem sempre têm os mesmos itens para todas as empresas, podendo ser adequados conforme as necessidades, o foco e o público-alvo, porém devem deixar bem claro “o quê”, “como” e “quando” serão feitos os testes e o motivo pelo qual a sua existência e as etapas que o compõem são importantes.

Itens de um plano de teste



Como indicado anteriormente, os planos de teste diferem conforme o projeto, mas é de extrema importância conhecer os principais itens que devem existir nesse documento. Com base nesse modelo que será apresentado, é possível incluir ou excluir elementos para adequar aos seus futuros projetos.

Itens que podem aparecer em um plano de teste:



a) Nome e resumo do sistema a ser criado

Aqui, o mais importante é uma explicação clara do que será o sistema, para que qualquer pessoa que leia o documento pela primeira vez saiba identificar o que ele faz e quais são as suas funcionalidades.



b) Objetivos do plano de teste

Aqui, deve ser citado tudo o que se espera de resultados com a aplicação desse plano de teste.



c) Escopo do projeto

Neste item, deve-se especificar os documentos que serão utilizados para a criação do plano de teste, por exemplo: manual de requisitos, diagramas, regras de negócios, documento de arquitetura, lista de riscos, protótipo, entre outros.



d) Tipos de teste

Este é um dos itens mais importantes de um plano de teste, pois aqui são definidos todos os elementos do sistema que serão testados. Uma falha nesta fase do documento pode acarretar sérios riscos ao futuro do projeto. Por exemplo: teste de banco de dados, teste de interface, teste de carga, teste de *performance* etc.



e) Requisitos de teste

Este item mostra os casos de uso e os requisitos funcionais e não funcionais que serão testados por cada tipo de teste escolhido para esse projeto.



f) Estratégias de teste

Aqui, é especificado o objetivo de cada um dos testes, qual será o seu modo de utilização, por quem será feito o teste, os critérios de aceitação dos resultados e a especificação de infraestrutura necessária.



g) Ferramentas

Aqui, devem ser citadas as ferramentas utilizadas para os testes e suas respectivas etapas. É preciso citar, caso necessário, as ferramentas utilizadas para os testes e em qual etapa de teste ela será empregada.

h) Equipe

Aqui, devem ser indicados todos os integrantes da equipe de teste, especificando a função de cada um no projeto e em quais testes terão participação.



i) Infraestrutura de teste

Aqui, deve-se listar tudo que é necessário de *hardware* e *software* para as fases de teste do projeto, inclusive especificando versões e marcas, se necessário.



j) Cronograma

Neste item, listam-se os prazos de execução de cada etapa do projeto.



k) Modelo de reportagem de erros

Neste item, deve-se indicar a forma como serão apresentados os erros que aparecerem nos testes.

l) Avaliação dos resultados

Com base nos resultados obtidos, são tomadas as decisões para o andamento do projeto, que podem ser: recomeçar algo que não está dando certo, atualizar parte do sistema ou manter o que está funcionando dentro do esperado.

Etapas de criação de um plano de teste e tipos de testes

Para começar o exemplo de um plano de teste, simule um sistema simples de cadastro de usuários e *login*, em que serão criadas algumas regras a serem seguidas durante o processo de cadastro de novos usuários e haverá todos os testes necessários para analisar o funcionamento do sistema.

Figura 1 – Tela do sistema

Fonte: adaptado de Figma (2023)

1. Nome e resumos do sistema

O sistema é chamado “Login +” e pode ser utilizado como um *script* de *login* acoplado a outros sistemas. Ele tem duas telas em que o usuário pode cadastrar seu *login* e uma tela para o *login* ser executado. Esse *script* serve para aumentar a segurança de um sistema que não apresente essa funcionalidade.

2. Objetivos do plano de teste

Esse documento do plano de testes do sistema Login + é composto dos seguintes objetivos:

Identificar os tipos de dados que devem ser aceitos pelo cadastro no banco de dados.

Recomendar estratégias de teste que funcionam melhor no sistema Login +.

Listar a forma de apresentação dos resultados dos testes.

Listar a equipe de pessoas responsável pelos testes.

Criar o cronograma de testes.

3. Escopo do projeto

O plano de teste engloba os setores de banco de dados, telas de cadastro e *login*, segurança/criptografia da senha, análise do protótipo e interface do projeto.

A interface será testada apenas para sistemas *desktop* e inclui as telas de *login* e cadastro, além das mensagens de erro.

Todos os testes são finalizados quando cada uma das etapas estiver funcionando.

Tipos de testes a serem executados

É preciso descrever os tipos de testes que farão parte do plano e qual será o seu objetivo nesse plano.

1. Tipos de testes

Todos os testes têm um objetivo, portanto a escolha deve ser bem elaborada, pois eles que cobrirão as diversas possibilidades de erros que podem ocorrer em um sistema. A seguir, veja o que se teria no plano de testes:

Os testes utilizados no projeto serão: teste estrutural, teste de banco de dados, teste funcional, teste de interface do usuário, teste de carga e teste de segurança.

2. Requisitos de teste

Nesta etapa, é preciso descrever o que será testado em cada um dos testes utilizados no sistema de cadastro e *login*.

2.1. Teste de banco de dados

Verifica se ocorre a conexão com o banco de dados.

Verifica se a senha cadastrada está criptografada conforme o método MD5.

Verifica se é possível cadastrar novos usuários no banco de dados.

Verifica se é possível testar os dados de *login* com os dados inseridos no banco de dados.

2.2. Teste funcional

Verifica se o programa inicia e finaliza corretamente sempre que o usuário desejar.

Verifica se as mensagens de *login* correto ou de erro de *login* aparecem quando deveriam.

Verifica se todas as funções do programa, neste caso, cadastro e *login*, estão funcionando.

Testa se o comportamento do sistema é o esperado.



Teste funcional é o conhecido como caixa-preta, em que não se tem acesso ao código-fonte, mas são testadas as diversas funções do sistema, com o máximo possível de variações de dados inseridos, para testar, por exemplo, as características a seguir:

Funções incorretas ou ausentes

Erros de desempenho

Erros de interface

Erros em acessos a bancos de dados externos

Erros de inicialização e término

Erros nos tipos de dados aceitos em campos de inserção

Aqui, é possível citar duas ferramentas principais de testes funcionais: Selenium e JUnit.

2.3. Teste de interface do usuário

Navega por todas as funcionalidades do programa, testando seu funcionamento.

Verifica se a interface é de fácil utilização e aprendizado.

Verifica se a interface mantém alinhamentos e proporções estipulados no protótipo.

Verifica se todo o texto apresentado na interface está conforme as normas de grafia e as regras gramaticais.

2.4. Teste de carga

Verifica o sistema com 1 usuário logado.

Verifica o sistema com 10 usuários logados.

Verifica o sistema com 20 usuários logados.

Verifica o sistema com 50 usuários logados.

2.5. Teste de segurança

Verifica se somente um usuário cadastrado tem acesso ao sistema.

Verifica se não é possível acessar a base de dados por meio das caixas de texto do sistema, por comando SQL (*structured query language*).

Como optou-se por um exemplo simples para facilitar o entendimento, serão feitos apenas esses testes, mas seria possível ter, por exemplo, teste automatizado, teste de desempenho, teste de volume etc.

3. Estratégias de teste

Agora, saiba mais sobre as estratégias de teste.

3.1. Teste de banco de dados

Objetivo do teste	Garantir que os dados cadastrados estejam sendo salvos corretamente no banco de dados e que o sistema de <i>login</i> consiga acessar a base de dados para autorizar o usuário.
Técnica	Invocar cada método de cadastro ao banco de dados com valores válidos e inválidos. Testar o processo de consulta à base de dados por meio do <i>login</i> .
Critério de finalização	Todos os acessos à base de dados funcionam como projetados e sem perda de informações.
Considerações especiais	Processos devem ser invocados manualmente.

Tabela 1 – Teste de banco de dados

Fonte: Senac EAD (2023)

3.2. Teste funcional

Objetivo do teste	Garantir que as principais funções do programa estejam funcionando.
Técnica	Executar todos os casos de uso, usando dados válidos e inválidos, para verificar se: Cada regra de negócio é aplicada corretamente. As mensagens de erro aparecem sempre que necessário e da forma como foram projetadas. Quando dados válidos são colocados, ocorrem os resultados esperados.
Critério de finalização	Todos os testes foram aplicados com êxito. Todos os erros identificados foram corrigidos ou reportados.
Considerações especiais	Aqui, pode ser reportado algo que ajude na avaliação final dos testes funcionais.

Tabela 2 – Teste funcional

Fonte: Senac EAD (2023)

3.3. Teste de interface do usuário

Objetivo do teste	Garantir que a interface apresente o mesmo formato projetado no protótipo e respeite as regras de usabilidade definidas pela equipe de desenvolvimento e seus gerentes.
Técnica	Testar todos os elementos de cada janela da interface, especialmente botões, textos e mensagens de erros. Modificar o tamanho das janelas para ver se funcionam nos dispositivos para os quais foram projetadas.
Critério de finalização	As janelas funcionam sem erros em todos os dispositivos para os quais foram projetadas. Botões e mensagens de erro, ou avisos, abrem corretamente sempre que for necessário. A interface respeita as regras de usabilidade do projeto do sistema.
Considerações especiais	Nenhuma

Tabela 3 – Teste de interface do usuário

Fonte: Senac EAD (2023)

3.4. Teste de carga

Objetivo do teste	Testar o tempo de resposta do sistema, alterando o número de usuários acessando ao mesmo tempo.
Técnica	Testar o tempo de resposta do sistema, alterando o número de usuários acessando ao mesmo tempo.
Critério de finalização	O sistema não apresentar falhas nem perda de acesso com até 500 usuários ao mesmo tempo.
Considerações especiais	Deve-se ter toda a base de dados em uma mesma máquina recebendo todos os acessos para que o teste tenha validade.

Tabela 4 – Teste de carga

Fonte: Senac EAD (2023)

3.5. Teste de segurança

Objetivo do teste	Verificar se somente usuários cadastrados têm acesso ao sistema. Verificar se as senhas estão criptografadas na base de dados.
Técnica	Testar dados válidos e inválidos para tentar acessar o sistema e testar se os dados de cadastro estão criptografados na base de dados.
Critério de finalização	O teste é finalizado quando todos os acessos válidos funcionam e não é encontrado nenhum acesso inválido no processo.
Considerações especiais	Nenhuma

Tabela 5 – Teste de segurança

Fonte: Senac EAD (2023)

Agora, veja o desafio a seguir.

Agora é hora de um pequeno desafio!

Desafio 1: Imagine um sistema que recebe o nome do aluno e duas notas desse aluno, calcula a sua média e, ao fim, grava, em uma base de dados, o nome, a média e se o aluno foi aprovado com base em uma média maior ou igual a 7.

Defina que tipos de testes você usaria nesse sistema e qual estratégia de teste seria utilizada.

4. Equipe

Veja a tabela a seguir.

Cargo	Responsabilidades
Gerente	Define a equipe de teste Indica direcionamento de funções Fornece relatórios Adquire recursos necessários
<i>Test designer</i>	Identifica, prioriza e implementa os casos de teste Gera o plano de teste Cria o modelo de cada teste
<i>Designer gráfico/visual</i>	Responsável pela elaboração dos testes de interface
Testador	Executa os testes Registra os resultados Documenta e solicita mudanças
Gerente de banco de dados	Prepara o banco de dados para os testes Administra os dados durante os testes
Implementador	Implementa as classes de teste

Tabela 6 – Equipe

Fonte: Senac EAD (2023)

5. Infraestrutura de teste

Aqui, são colocados todos os *hardwares* e *softwares* necessários para o bom andamento da fase de testes.

Recursos de infraestrutura
Servidor de banco de dados MySQL
NetBeans 13
Terminais de teste Três PCs (computadores pessoais) – não é necessária conexão à Internet

Tabela 7 – Recursos de infraestrutura

Fonte: Senac (2023)

6. Cronograma

Aqui, será simulado um cronograma de atividades de teste. É sempre importante ter um último item de avaliação dos testes, pois decisões são tomadas com base nos resultados, devendo, inclusive, ser criado um novo cronograma de testes sempre que houver atualizações.

Etapas	Data de início	Data de término
Planejar teste	20/05/2023	22/05/2023
Teste estrutural	22/05/2023	22/05/2023
Teste de banco de dados	23/05/2023	23/05/2023
Teste funcional	24/05/2023	24/05/2023
Teste de interface	25/05/2023	25/05/2023
Teste de carga	26/05/2023	26/05/2023
Teste de segurança	27/05/2023	27/05/2023
Avaliar teste	28/05/2023	30/05/2023

Tabela 8 – Cronograma

Fonte: Senac EAD (2023)

Especificação, relato e registro de teste

Esta parte do plano de teste é de extrema importância, pois especifica a forma como os erros serão reportados e qual será a avaliação dos resultados e das medidas futuras a serem tomadas.

7. Modelo de reportagem de erros

Na parte de reportagem de erros, utiliza-se um programa do tipo *bugtracker* (específico para reportar os erros).

Um dos modelos mais aceitos para a descrição de erros é o que utiliza três itens para citar cada erro existente. Veja a seguir.

a. Sumário

É onde se coloca alguma abreviação, caso haja, com o nome do erro. Por exemplo:

- *[BUG] Erro de cadastro no campo **E-mail**.*

b. Descrição

É onde vai a mensagem apresentada pelo programa, ou a descrição do erro feita pelo testador, caso o sistema não apresente um erro aparente, mas a pessoa responsável pelo teste entenda que algo deveria ter acontecido diferente do que aconteceu.

- *O cadastro não apresentou erro ao ser inserido um e-mail sem “@”.*

c. Informação adicional

É qualquer informação adicional que possa ajudar o gerente de teste nas tomadas de decisões posteriores, como uma possível solução ao erro, uma sugestão de usabilidade ou até o que imagina ser a origem do erro.

- *Precisa ser usada uma validação de formulários. Sugere-se o uso de expressões regulares para a solução do problema.*

d. Prioridade

Permite definir níveis de importância para os erros, que podem ser, por exemplo, baixa, média e alta.

- *Prioridade alta*

8. Avaliação dos resultados do teste

Nesta etapa, analisam-se os resultados obtidos após todas as fases de testes, normalmente criados pelo testador, que, evidentemente, pode ser de mais de uma área (programador, *design*, usuários do público-alvo do projeto). Com base nas respostas registradas pelos testadores, decide-se o que deve ser feito.

A seguir, veja outro desafio.

Que tal um pequeno desafio?

Desafio 2: Com base no desafio 1, simule possíveis erros que podem aparecer e use o método de reportagem aprendido para descrever esses erros. Ao final, crie uma avaliação dos resultados, com possíveis medidas a serem tomadas com base nos erros apresentados.

Encerramento



Após todas as etapas do plano de teste finalizadas, fica evidente a importância de um documento de testes muito bem elaborado, que abranja o máximo das possibilidades que podem aparecer durante um processo de criação de um sistema e, depois, durante toda a vida útil dele. Esse documento planeja um caminho que deve ser seguido pela equipe de testes e evita o retrabalho de ir descobrindo o que deve ser testado conforme os erros forem aparecendo.

Sendo assim, essa etapa não pode ser ignorada, pois qualquer erro futuro que não foi projetado pode colocar em risco todo o andamento do processo e o prazo de conclusão do seu *software*. Dessa forma, entende-se que um documento bem formatado agiliza o trabalho da

equipe e deixa mais rápido o processo de criação do sistema. Além disso, esse documento deixa o sistema com uma base muito mais sólida para enfrentar os desafios de mercado, pois os principais erros que poderiam ocorrer já foram tratados.